B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

HEXplorer

By
Tiffany Ngik Chee Yong

April 26, 2026

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **2**
1.1 The Problem: Your Data Is Never Truly Private 2
1.2 The Solution: Computing on Locked Data 2
1.3 Real-World Applications 3
1.4 Project Overview: HEXplorer 4
1.5 Project Objectives 5
1.6 Why This Matters 6
1.7 Project Contributions 6
1.8 Report Structure 7

2 Methodology **8**
2.1 Development Approach 8
2.2 Requirements Gathering 8
2.3 Iterative, Phase-Based Development 9
2.4 Project Management 11
2.5 Validation and Testing 12
2.6 Empirical Approach and Problem Solving 13

3 Technology Review **16**
3.1 The Benchmarking Gap 16
3.2 Homomorphic Encryption 18
3.2.1 Significance 18
3.3 BFV, BGV, and CKKS: HE Scheme Families 19
3.4 Microsoft SEAL 20
3.5 IBM HElib 21
3.6 OpenFHE 21
3.7 Comparison of HE Libraries: SEAL, HElib, and OpenFHE 22
3.8 Rust 22
3.9 gRPC and Protocol Buffers 23
3.9.1 Comparison: gRPC vs REST 23
3.10 Spring Boot 25

3.11 React . . . 25
3.12 Docker and Containerisation . . . 26
3.13 PyTorch and CNN Training . . . 27
3.14 AWS EC2 and Vercel . . . 28
3.15 Chapter Summary . . . 29

4 System Design **31**
4.1 System Overview . . . 31
4.2 Layered Architecture . . . 32
4.2.1 Presentation Tier: React and Nginx . . . 36
4.2.2 Application Tier: Spring Boot . . . 40
4.2.3 Compute Tier: Rust gRPC Server and HE Wrappers . . . 40
4.3 Communication and Data Flow . . . 44
4.3.1 Primitive Benchmark Flow . . . 44
4.3.2 Synchronous Inference Flow . . . 45
4.3.3 Streaming Inference Flow . . . 46
4.4 Design Decisions . . . 48
4.5 Deployment Architecture . . . 49
4.5.1 Cloud Infrastructure Overview . . . 49
4.5.2 EC2 Instance Types and Rationale . . . 50
4.5.3 Network Security . . . 50
4.5.4 Why This Design Is Sound . . . 51
4.6 Chapter Summary . . . 52

5 System Evaluation **54**
5.1 Evaluation Overview . . . 54
5.2 Evaluation Against Objectives . . . 55
5.2.1 O1 — End-to-End Encrypted Inference . . . 55
5.2.2 O2 — Accuracy Parity with Plaintext . . . 55
5.2.3 O3 — Functional Cross-Library Integration . . . 56
5.2.4 O4 — Cross-Library Performance Comparison . . . 56
5.2.5 O5 — Real-Time Observability . . . 57
5.3 Performance Results . . . 57
5.3.1 Inference Latency . . . 57
5.3.2 Per-Layer Latency Breakdown . . . 59
5.3.3 Model Training Overview . . . 60
5.3.4 Accuracy vs Activation Degree . . . 60
5.3.5 Cross-Library Primitive Benchmarks . . . 61
5.4 Analysis and Discussion . . . 63
5.4.1 Inference Accuracy . . . 63
5.4.2 Inference Latency . . . 63

5.4.3 Library Comparison 63
5.4.4 Cold-Start and Memory 63
5.4.5 Key Findings 63
5.5 System Robustness 64
5.5.1 Testing Results 64
5.5.2 Stability Observations 64
5.6 Limitations 65
5.7 Chapter Summary 65

6 Conclusion 67
6.1 Project Summary 67
6.2 Key Findings 67
6.3 Insights and Reflection 69
6.4 Limitations and Future Work 70
6.5 Closing Remarks 71

A GitHub Repository and Installation Instructions 72
A.1 Source Code Repository 72
A.2 Prerequisites 73
A.3 Quick Start: Docker Compose (Recommended) 73
A.4 Separate Frontend and Compute Deployment 74
A.5 Native Build (Without Docker) 75
A.6 Running the MNIST Training Scripts 76
A.7 Verifying the Installation 77

List of Figures

1.1 Overview of the three modes of HEXplorer. 5

2.1 Six development phases and their dependencies. 10
2.2 Commit activity over the project timeline. 12
2.3 GitHub Actions CI pipeline — all four jobs passing. 14

3.1 Capability matrix comparing HEBench, CryptoNets, and HEXplorer across seven benchmark-relevant features. 17
3.2 BFV noise budget across the twelve-layer CNN pipeline: conservatively sized parameters (top) versus an under-sized configuration that fails before decryption (bottom). 19
3.3 Data flow for a `RunBenchmark` gRPC call, from browser request to per-operation timing result. 24
3.4 Multi-stage Dockerfile for HEXplorer: three HE library builder stages plus a Rust builder feeding a lean runtime image. 27

4.1 UML component diagram (coarse grain): the five HEXplorer components and their interface protocols. 33
4.2 AWS VPC deployment topology for HEXplorer. 34
4.3 Overall HEXplorer system architecture. 37
4.4 Digit inference canvas: the user draws a digit, which is encrypted and sent to the server for homomorphic inference. 38
4.5 Loading animations used during server-side HE computation: the owl (inference) and the rotating wireframe cube (benchmarks). . . . 39
4.6 Benchmark results UI: primitive operation timings for the selected HE library rendered as a bar chart. 39
4.7 UML class diagram (fine grain): internal structure of the Rust compute tier. `HEServiceImpl` dispatches requests through `SessionStore` to a concrete `HEWrapper` implementation. 43

4.8 Sequence diagram for a `RunBenchmark` request. Each primitive operation is timed independently by the Rust harness before results are returned up the stack. 45
4.9 Sequence diagram for a `PredictDigitStream` request. Per-layer progress events propagate from the Rust server through Spring Boot SSE to the browser in real time. 47

5.1 Where time goes during one BFV inference (degree-2, 128-bit). The two convolution layers and the fully-connected layer together take up almost 88% of the total time. 59
5.2 Primitive operation timings for all three libraries (log scale, 128-bit security, `r6i.large`). 62

List of Tables

2.1 Supervisor meeting log mapped to development phases. 13
2.2 Spring Boot controller test coverage. 14

3.1 Comparison of the three HE libraries integrated into HEXplorer. . . 22
3.2 HEXplorer technology stack: role and selection rationale. 30

4.1 HEXplorer components and their inter-component dependencies. . . 32
4.2 Spring Boot REST endpoints and their gRPC counterparts. 40

5.1 Encrypted CNN accuracy on ten MNIST images by activation degree (OpenFHE BFV, 128-bit security, scale factor 1 000). 56
5.2 Primitive operation timings per library (128-bit security, $n = 10$ ops, `r6i.large`). All values in milliseconds. 56
5.3 Per-image inference latency, degree-2 activation, 128-bit security, `r6i.large`. 58
5.4 Average per-layer latency across ten test images (degree-2 activation, 128-bit security). Percentages are of the mean total time (13 919 ms). 59
5.5 Accuracy and average inference latency by activation degree (OpenFHE BFV, 128-bit, ten test images, `r6i.large`). 61

A.1 Required software and minimum versions. 73

Chapter 1

Introduction

1.1 The Problem: Your Data Is Never Truly Private

Imagine you are a patient with a rare skin condition. Your doctor wants to run an AI model to help with the diagnosis. To do that, the model needs your medical scans — and the scans need to be sent to a remote server where the AI runs. The moment your data leaves your device, someone else can see it. Even if the company promises to keep it safe, your private health information is now sitting on a server you do not control.

This is not a hypothetical problem. In 2023, Samsung engineers accidentally exposed proprietary source code by pasting it into an AI tool [1]. A misconfigured cloud storage bucket leaked patient scans from a medical AI startup, violating privacy regulations [2]. These incidents share the same root cause: traditional encryption protects your data *while it travels*, but the server must *decrypt* it before it can do anything useful. Decryption means exposure.

Regulations like GDPR [3] and the EU AI Act [4] now impose heavy fines for data misuse, but they cannot fix the underlying technical problem: *if you want a remote server to analyse your data, the server has to see your data.*

Or does it?

1.2 The Solution: Computing on Locked Data

There is a branch of cryptography called **Fully Homomorphic Encryption** (FHE) that flips this assumption. FHE lets a server run computations — like classifying a medical image — directly on *encrypted* data, without ever unlocking it. The server sees only scrambled numbers throughout. Only the person who

holds the key can read the final result.

Think of it like a locked puzzle box. You put your data inside and lock it. You hand the box to a machine. The machine does its work on the outside of the box — rearranging, adding, multiplying — and hands the box back. You unlock it and see the answer. The machine never saw what was inside.

This idea was first proposed as a theory in 1978 [5] and only became mathematically possible in 2009, when Craig Gentry proved it could be done in his landmark PhD dissertation [6]. Since then, researchers at Microsoft, IBM, Google, and Intel have been working to make it fast enough to use in practice [7, 8, 9].

Today, FHE is still slow — running an AI model on encrypted data can take tens of seconds instead of milliseconds. **Understanding exactly how slow, and why, is what this project is about.**

1.3 Real-World Applications

FHE is not just a theoretical curiosity. Researchers have demonstrated practical applications across several high-stakes domains:

- **Healthcare and medical imaging.** Soni and Kumar [10] demonstrated FHE-based secure aggregation of medical data in mobile edge computing, published in the *IEEE Internet of Things Journal*. The same principle applies to AI diagnostics: a hospital sends encrypted patient scans to a remote AI service, receives an encrypted prediction, and decrypts it locally. The AI company never sees the patient's data [11].

- **Genomics and biometrics.** Blatt et al. [12] performed genome-wide association studies on encrypted DNA data, published in *PNAS*. This shows that FHE can protect the most sensitive personal data of all — your genetic code — while still allowing meaningful analysis.

- **Finance and fraud detection.** Boura et al. [13] showed how deep learning predictions can be evaluated homomorphically for financial applications. A bank could run fraud detection on an encrypted transaction without the fraud-detection service ever learning the transaction amount or who made it [14].

- **Federated machine learning.** Zhang et al. [15] proposed BatchCrypt, which uses HE to protect model updates in cross-organisation federated learning. This means multiple hospitals or banks can train a shared AI model without sharing their raw data with each other.

- **Privacy-preserving AI in general.** A 2021 survey in the *IEEE Transactions on Knowledge and Data Engineering* [16] identified FHE as one of the three core building blocks for privacy-preserving machine learning, alongside differential privacy and secure multi-party computation.

- **Cloud computing.** A company could outsource heavy computation to an untrusted cloud provider while keeping their data encrypted throughout, as formalised in the NIST cloud security model [17, 6].

The barrier to adoption is not that FHE is theoretically unsound — the papers above prove it works. The barrier is that the performance costs are not well-understood outside specialist circles, and there is no accessible tool for measuring them. Before deploying FHE in any of the settings above, developers need to know: how long does it take? Which library is fastest? How does security level affect memory usage? These questions do not have easy answers today.

1.4 Project Overview: HEXplorer

HEXplorer is an interactive platform built to answer those questions. Think of it as a *benchmarking playground* for homomorphic encryption — where anyone with a browser can run real encrypted computations, watch the results appear live, and compare three different FHE libraries side by side. No installation needed. No command line. Just open a browser and go.

The platform has three modes, like three levels of a game:

Level 1 — Draw and Predict. You draw a handwritten digit (0–9) in your browser. HEXplorer encrypts your drawing, sends it to a server, runs it through a neural network *entirely on encrypted data*, and sends back the prediction. The server never sees your pixel values. You watch each layer of the network complete in real time.

Level 2 — Batch Test. Ten standard test images are sent through the encrypted pipeline one by one. HEXplorer measures how long each image takes, checks whether the prediction is correct, and displays a per-layer breakdown of where the time goes. This gives a rigorous measure of how well encrypted AI actually works.

Level 3 — Library Race. Three FHE libraries — Microsoft SEAL, IBM HElib, and OpenFHE — are run head-to-head on five basic operations: generating keys, encrypting data, adding two encrypted values, multiplying two encrypted values,

and decrypting the result. A bar chart updates live as each library finishes. You can see instantly which library is fastest and by how much.

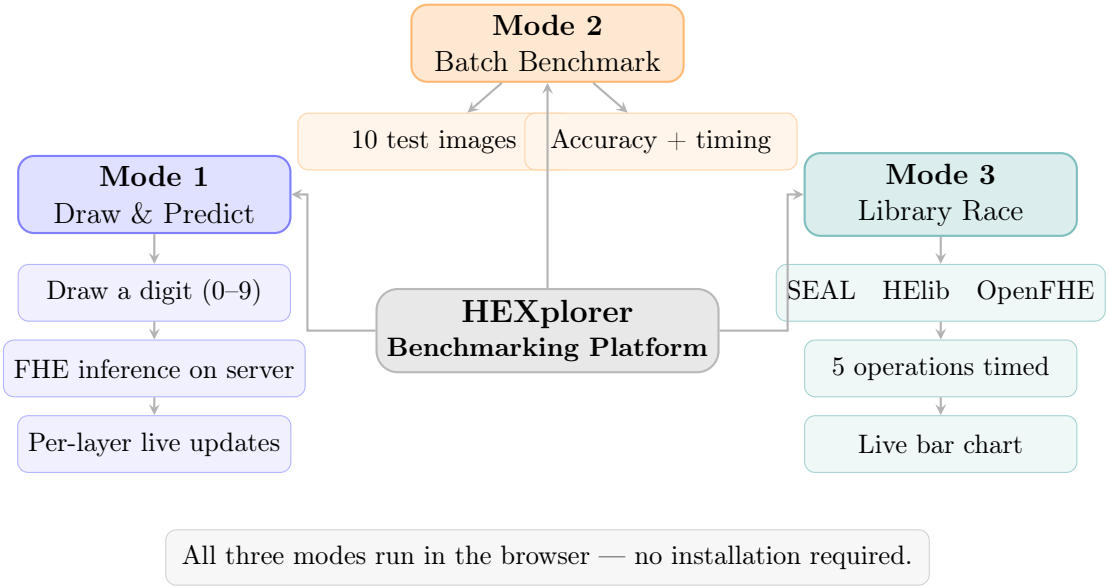

Figure 1.1: Overview of the three modes of HEXplorer.

Figure 1.1 gives an overview of the three modes that make up HEXplorer. Together they cover end-to-end encrypted inference, accuracy measurement, and cross-library performance comparison. Mode 1 runs live encrypted inference on a user-drawn digit. Mode 2 batch-tests ten images and records accuracy and per-layer timing. Mode 3 benchmarks three FHE libraries head-to-head on five operations and displays results as a live bar chart.

1.5 Project Objectives

The project is measured against five specific objectives. Each one has a clear, testable success criterion that is evaluated in Chapter 5.

- **O1 — Encrypted inference works end-to-end.** The full CNN pipeline must run on encrypted data from start to finish. The server must never see any pixel values in plaintext. *Success: all ten test images return a correct result with status "success".*
- **O2 — Encrypted model matches plaintext accuracy.** The encrypted pipeline must achieve the same classification accuracy as running the same

model without encryption, on the same ten test images. *Success: 10/10 correct at the reference configuration (degree-2, 128-bit, scale 1 000).*

- **O3 — All three libraries are integrated and working.** SEAL, HElib, and OpenFHE must all run inside the same system and each must return timing results for five operations. *Success: benchmark endpoint returns valid timings for all three libraries.*

- **O4 — Results are comparable in the browser.** A user must be able to compare all three libraries side by side in a bar chart, without installing anything. *Success: the Library Comparison page renders results for all three libraries.*

- **O5 — Progress is shown live, layer by layer.** The browser must update each CNN layer's timing as it finishes, not all at once after inference ends. *Success: each layer row updates in real time during inference.*

1.6 Why This Matters

Several research groups have demonstrated that FHE-based inference is *possible* [11, 18, 19]. The original CryptoNets paper [11] showed in 2016 that a neural network could classify encrypted MNIST digits with over 98% accuracy. More recent work such as Gazelle [18] and Cheetah [19] has pushed latency down using hybrid cryptographic protocols.

What is missing is a *platform* that makes these results accessible and reproducible. Existing tools like HEBench [20] require local installation, produce raw CSV output, and benchmark only primitive operations — not end-to-end inference. No existing open tool lets a non-expert trigger encrypted inference in a browser and observe the cost of each layer in real time.

HEXplorer fills that gap. It is not just a benchmark — it is an educational and exploratory tool that makes the practical cost of privacy-preserving AI tangible to anyone.

1.7 Project Contributions

1. **End-to-end encrypted CNN inference platform.** A complete pipeline from browser input to encrypted prediction, using OpenFHE BFV, achieving 100% accuracy on ten MNIST test images at 128-bit security.

2. **Unified cross-library benchmarking.** SEAL, HElib, and OpenFHE are evaluated on identical workloads within one integrated system, producing directly comparable timing data.

3. **Real-time observability via SSE streaming.** Per-layer inference timings are streamed live to the browser as each layer completes, making bottlenecks visible without post-processing.

4. **Reproducible Docker environment.** All three HE libraries are compiled from source in a multi-stage Docker build, reducing setup from several hours to under five minutes.

The full source code is available at:
`https://github.com/TiffanyYongNgikChee/grpc-he-benchmark`
The demo is available at:
`https://youtu.be/-hU0B2686-g?si=2xzZ0W63JGrHHRkk`

1.8 Report Structure

The remainder of this dissertation is organised as follows:

- **Chapter 2 — Technology Review.** Reviews the relevant background: how FHE works, what BFV, BGV, and CKKS are, how neural networks run on encrypted data, and what existing tools and papers say about performance.

- **Chapter 3 — Methodology.** Describes how the project was managed and built: the six development phases, requirements gathering, testing strategy, and how problems were discovered and solved.

- **Chapter 4 — System Design.** Explains the architecture of HEXplorer: how the React frontend, Spring Boot API, gRPC server, Rust inference engine, and three C++ HE libraries fit together.

- **Chapter 5 — System Evaluation.** Tests the system against O1–O5. Presents inference latency, per-layer breakdown, accuracy by activation degree, cross-library timing comparison, and robustness results.

- **Chapter 6 — Conclusion.** Summarises the key findings, reflects on what was learned, and identifies directions for future work.

Chapter 2

Methodology

2.1 Development Approach

This project had two goals running in parallel: building a working system *and* producing reliable research results. A standard software engineering approach alone would not guarantee fair experimental results; a pure research approach would not address the complexity of building and deploying the system.

A **hybrid methodology** was therefore adopted:

- **Agile, phase-based engineering** — the system was built incrementally. Each phase produced a tested, working component before the next phase began, so problems were caught early.
- **Structured benchmarking** — all experiments used the same hardware, the same security parameters, and the same measurement method (wall-clock time in Rust) to keep results fair and repeatable.

2.2 Requirements Gathering

Requirements were gathered from three sources before any code was written.

Personal experience. Working directly with SEAL, HElib, and OpenFHE revealed how time-consuming the setup is and how difficult it is to compare libraries fairly. Raw timing numbers alone are hard to interpret. This motivated the need for a unified, visual benchmarking platform.

Review of existing tools. Tools such as HEBench and the built-in example suites were examined. They are limited to local environments, require significant manual setup, and produce outputs that are not easy to interpret. None are accessible through a browser. This confirmed a gap that HEXplorer could fill.

Project objectives (O1–O5). Each objective mapped directly to a system feature:

- O1 — end-to-end encrypted inference pipeline
- O2 — accuracy evaluation against a plaintext baseline
- O3 — benchmarking all three libraries under the same conditions
- O4 — side-by-side visual comparison in a browser
- O5 — real-time, layer-by-layer progress streaming

User Stories

To make the requirements concrete, a small set of user stories was written from the perspective of likely users:

- *"As someone curious about encryption, I want to draw a digit and see it classified by an encrypted model, so I can understand how FHE works without reading research papers."*
- *"As a student, I want to run test images through the encrypted pipeline and compare its accuracy to a normal model."*
- *"As a developer, I want SEAL, HElib, and OpenFHE benchmarked on the same tasks, so I do not need to set up each library separately."*
- *"As a user, I want to see each network layer process in real time, so I can understand where the bottlenecks are."*
- *"As someone reproducing the results, I want the system to start with a single Docker command."*

2.3 Iterative, Phase-Based Development

Development was split into six phases. Each phase depended on the previous one being stable — for example, the Spring Boot API could not be completed until gRPC communication was reliable, and benchmarking could not begin until the full inference pipeline was working end-to-end.

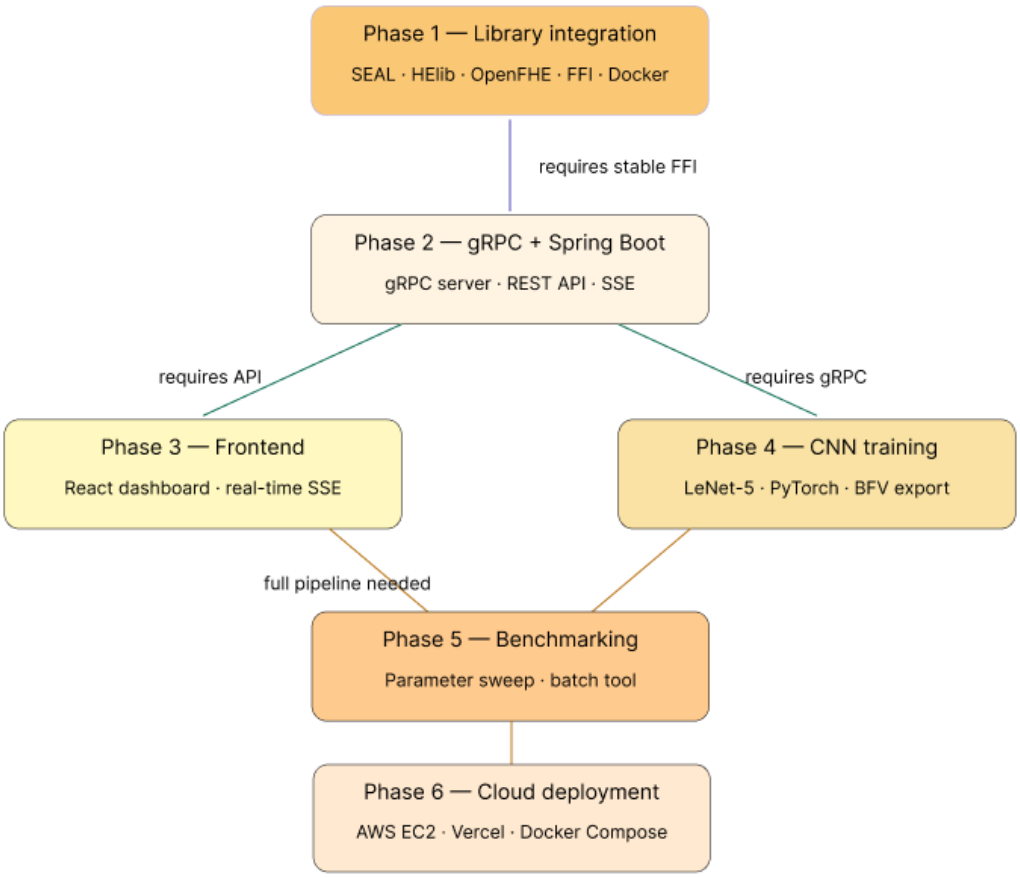

Figure 2.1: Six development phases and their dependencies.

1. **Phase 1 — Library integration and Rust core (Oct–Nov 2025).** SEAL, HElib, and OpenFHE were each integrated by writing Rust FFI bindings for the C++ code. A single common benchmarking interface was built so that any library could be swapped in without changing the rest of the system. Docker was introduced here to keep dependencies consistent across machines.

2. **Phase 2 — gRPC server and Spring Boot API (Dec 2025–Jan 2026).** A gRPC server exposed the encryption operations. A Spring Boot REST API was added as a middle layer between the backend and the browser, providing HTTP endpoints. Server-Sent Events (SSE) were wired up to support real-time progress streaming.

3. **Phase 3 — React frontend (Feb 2026).** A web dashboard was built and connected to the REST API. A progress tracker was initially driven by estimated timings, later replaced with live SSE updates once the backend streaming was stable.

4. **Phase 4 — CNN training and weight export (Mar 2026).** A LeNet-5 CNN was trained on MNIST in PyTorch using polynomial activations (x^2, x^3, x^4) instead of ReLU, which cannot be computed on encrypted data. Weights were exported and loaded into the Rust inference engine. The full pipeline — browser input to encrypted prediction — was tested end-to-end.

5. **Phase 5 — Parameter exploration and benchmarking (Late Mar–Apr 2026).** Experiments were run across different security levels, polynomial degrees, and scale factors. Results were collected using the batch benchmarking tool built into the dashboard. Memory exhaustion at 256-bit security and accuracy failures at degree-3 and degree-4 were recorded as findings.

6. **Phase 6 — Cloud deployment (Apr 2026).** The compute backend was deployed to AWS EC2 and the frontend to Vercel using Docker Compose. All final benchmark results in this dissertation were collected from this environment to ensure consistency.

2.4 Project Management

Three mechanisms kept the project on track.

Weekly supervisor meetings. Each session reviewed what had been completed, discussed any blockers, and agreed on the goal for the following week. Hav-

ing a concrete deliverable each week was a reliable way to keep progress steady even when technical problems were difficult.

GitHub Issues as sprint tickets. Each issue corresponded to a specific, testable task (e.g. *"Implement HElib key generation wrapper"*). Issues were tagged by component (`ffi`, `grpc`, `frontend`, `benchmark`) and closed with a reference to the relevant commit, making it easy to trace any decision back to its implementation. The repository accumulated over 300 commits across the project.

Commit history as an audit trail. Figure 2.2 shows commit activity over the project timeline. The distribution reflects the phase structure: heavier activity during integration and evaluation, lighter during planning and write-up.

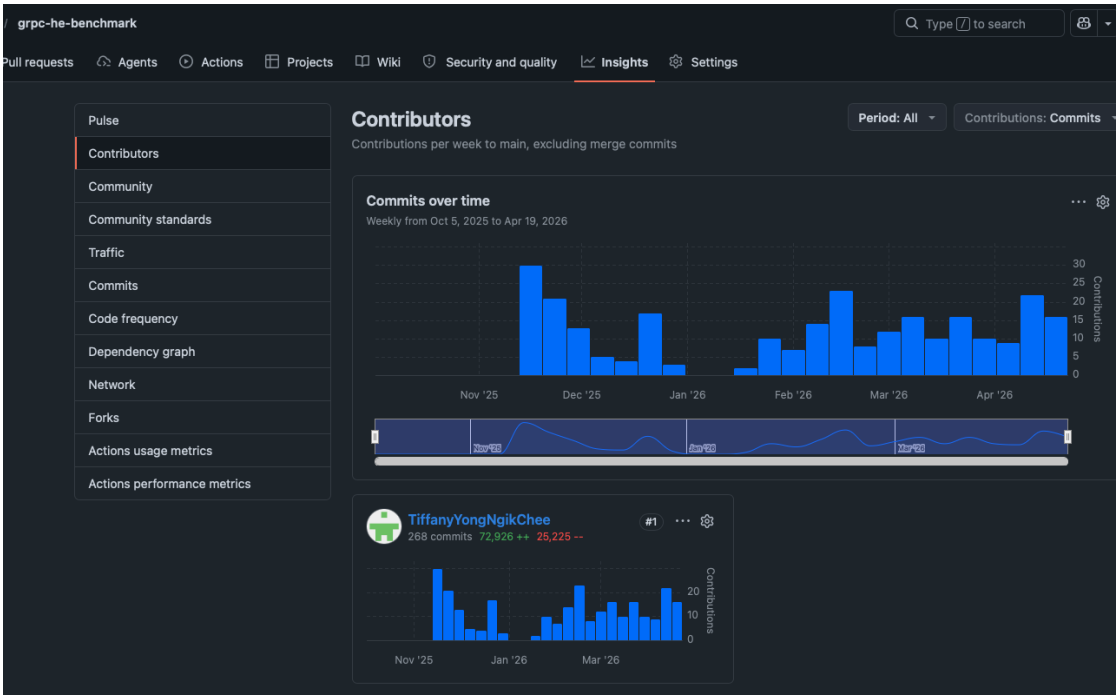

Figure 2.2: Commit activity over the project timeline.

2.5 Validation and Testing

Testing was structured across four layers of the stack.

Rust unit tests. Using `cargo test`, tests covered the full BFV lifecycle: context creation, key generation, and an encrypt–decrypt round-trip on a known vector. The weight loader was also tested separately to catch shape mismatches

Table 2.1: Supervisor meeting log mapped to development phases.

Period	Phase	Key topics
Oct 2025	Phase 1	Library selection; initial C++/Rust FFI
Nov 2025	Phase 1	Completed all three integrations; standardised interface
Dec 2025–Jan 2026	Phase 2	gRPC + REST API; SSE streaming design
Feb 2026	Phase 3	Frontend connected; progress tracker prototype
Mar 2026	Phase 4	CNN training; end-to-end inference pipeline
Late Mar–Apr 2026	Phase 5	Benchmarking; identified memory and accuracy limits
Apr 2026	Phase 6	Cloud deployment; final results collected

between the CSV exports from PyTorch and the tensor dimensions expected by Rust.

Spring Boot integration tests (JUnit 5). The REST API was tested with JUnit 5, MockMvc, and Mockito. The gRPC client was mocked so controller logic could be verified without a live backend. Table 2.2 shows the eight test cases.

Frontend simulation mode. The progress tracker had a simulation mode driven by hardcoded average layer timings. This allowed the full state machine (`idle` → `processing` → `done`) to be tested before the real streaming backend existed.

Continuous Integration (GitHub Actions). Figure 2.3 shows a CI pipeline ran on every push with four parallel jobs: frontend build and tests, Spring Boot Maven build, Rust/HE core tests inside Docker (with cached library builds to avoid a 20-minute rebuild on each commit), and a Docker validation step. Regressions were typically caught within two minutes of each commit.

2.6 Empirical Approach and Problem Solving

Research Before Building

Before writing code, background reading was done in two areas. For the encryption side, the documentation for all three libraries was studied to understand parameter constraints (polynomial degree, coefficient modulus) that must be set correctly

Table 2.2: Spring Boot controller test coverage.

Endpoint	Scenario	Expected
`GET /api/health`	Returns `"OK"`	200
`POST /api/predict`	Valid 784-pixel input	200
	Fewer than 784 pixels	400
	Null pixel array	400
	Scale factor zero — defaults to 1 000	200
	gRPC server unreachable	500
`POST /api/benchmark/run`	Valid SEAL request, timings returned	200
	gRPC call throws exception	500

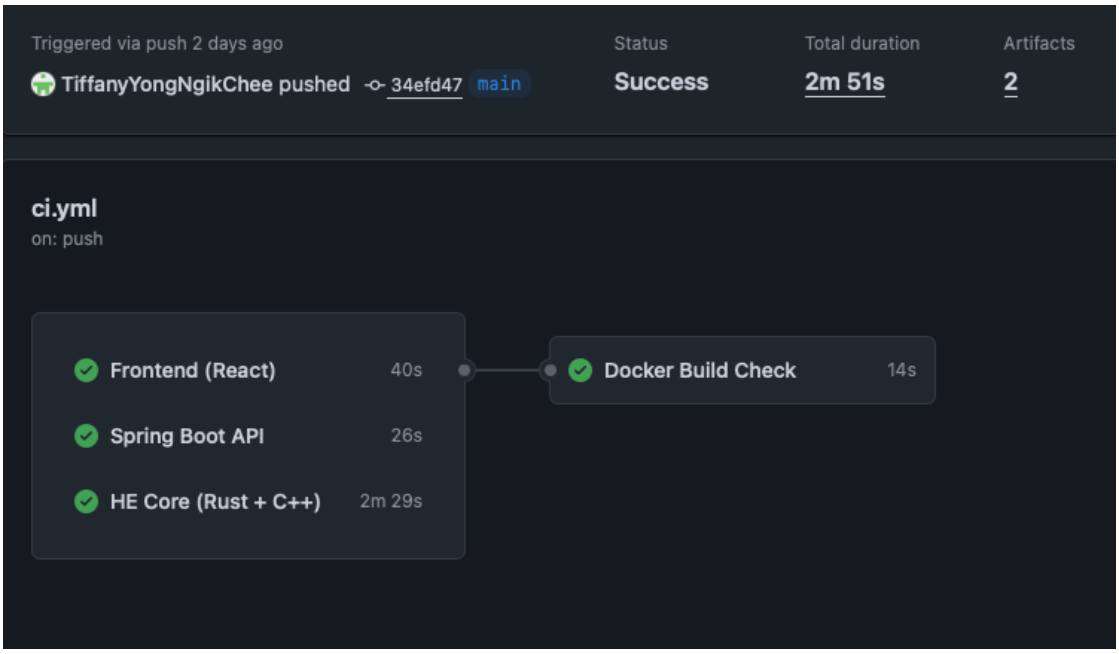

Figure 2.3: GitHub Actions CI pipeline — all four jobs passing.

before any computation. Wrong choices early on cause accuracy or noise problems that are very hard to trace later.

For the neural network side, existing work on FHE-compatible models confirmed that ReLU activations cannot be evaluated on encrypted data and that polynomial approximations are the standard solution.

Problems Encountered and How They Were Solved

Several issues were only discovered by running the system, not by planning:

- **Wrong output after decryption.** Early runs returned incorrect answers because noise from successive operations built up until the result was unreadable. Fix: adjusted encryption parameters to allow more computation headroom.
- **Out-of-memory at high security.** At 256-bit security the server crashed mid-run because ciphertexts are roughly 4× larger, exhausting the 16 GB instance. This was recorded as a finding rather than treated as a bug.
- **Inconsistent benchmark timing.** The first run of any benchmark was always slower than subsequent ones (a "warm-up" effect). Fix: added a discarded warm-up run before recording results.
- **API connection timeout during inference.** The Java–Rust connection dropped before inference finished (∼30–40 s) because the default HTTP timeout was too short. Fix: extended the timeout.
- **Live progress not streaming in the browser.** The first attempt used the browser's built-in `EventSource` API for SSE, which does not support a request body — but pixel data had to be sent in the body. Fix: switched to a `fetch`-based streaming reader that supports both.

Chapter 3

Technology Review

This chapter reviews the key technologies underpinning the HEXplorer system. For each technology the discussion covers: what it is, how it works at a conceptual level, why it is useful in general, and the specific rationale for its selection in this project. Where meaningful alternatives exist they are compared.

3.1 The Benchmarking Gap

Benchmarking homomorphic encryption is harder than it looks. HE performance is acutely sensitive to a constellation of interdependent parameters: ring dimension, plaintext modulus, activation polynomial degree, and noise budget all interact in ways that a single throughput number cannot capture. The most prominent community effort to address this is the HEBench framework [21], developed by Intel and collaborators to provide standardised workload specifications for HE library evaluation. HEBench is a meaningful step forward, but it has significant limitations for the scenario considered here.

It runs locally, sequentially, and produces static CSV output after completion. There is no client–server separation, no real-time observability, and no visual cross-library comparison. Integrating a new library requires manual CMake configuration that most practitioners find prohibitive. Research artefacts such as CryptoNets [11] and HETAL [22] have advanced inference latency considerably, but they fix the library and parameter set, require local compilation, and expose no accessible API.

Figure 3.1 summarises these capability gaps across three representative systems. HEBench supports configurable HE parameters but runs locally without a client–server boundary or visual interface. CryptoNets provides an end-to-end CNN inference demonstration but fixes both the library and the parameter set. HEXplorer is the only system in this comparison that combines remote execu-

tion, real-time per-layer observability, cross-library comparison, and a zero-install browser interface within a single reproducible environment. A ✓ indicates the capability is supported; a × indicates it is absent.

Capability	HEBench	CryptoNets	HEXplorer (This project)
Remote client–server execution	×	×	✓
Real-time per-layer observability	×	×	✓
Cross-library comparison (3 libs)	×	×	✓
Zero-install browser interface	×	×	✓
End-to-end CNN inference workload	×	✓	✓
Reproducible Docker environment	×	×	✓
Configurable HE parameters	✓	×	✓

Figure 3.1: Capability matrix comparing HEBench, CryptoNets, and HEXplorer across seven benchmark-relevant features.

This project extends the state of the art in three concrete ways. First, it deploys HE inference as a live cloud service with REST and gRPC APIs, enabling genuine client–server separation. Second, it provides side-by-side micro-benchmark comparison of Microsoft SEAL, IBM HElib, and OpenFHE within a single unified frontend, eliminating manual CSV collation. Third, it wraps the entire system in a reproducible Docker environment that reduces benchmark setup from hours to under five minutes.

The remainder of this chapter reviews each technology underpinning HEXplorer at a conceptual level, grounding every selection in authoritative sources and explaining why it was preferred over its alternatives. Section 3.2 covers Homomorphic Encryption theory; Section 3.3 examines the BFV, BGV, and CKKS scheme families; Sections 3.4–3.6 discuss the three HE libraries in detail, followed by a direct comparison in Section 3.7. Sections 3.8–3.14 then cover the infrastructure stack: Rust, gRPC and Protocol Buffers, Spring Boot, React, Docker, PyTorch, and the cloud deployment platforms.

Each section establishes the de facto or de jure standards in use, supports the methodology described in Chapter 2, and provides the technical foundation needed to interpret the evaluation results in Chapter 5.

3.2 Homomorphic Encryption

Homomorphic Encryption (HE) is a cryptographic technique that allows a server to run calculations on *encrypted* data without ever decrypting it. The idea was first posed as an open problem in 1978 [5] and only became mathematically realisable in 2009, when Craig Gentry published the first construction [6]. Surveys of the field can be found in Acar et al. [23] and Peikert [24].

How it works — the simple version:

- You encrypt your data and send the *locked* version to a server.
- The server does maths (add, multiply) directly on the locked data.
- You receive the locked result and decrypt it yourself.
- The server never saw your original numbers at any point.

Why it is hard — noise: Under the hood, HE is built on the Ring Learning With Errors (RLWE) problem [25, 14]. Every encryption wraps the data in a small amount of random noise. Each operation adds a little more noise. If you do too many operations the noise grows too large and decryption fails — like trying to read a message through too many layers of blurring. The remaining capacity before failure is called the *noise budget*. Parameter choices (ring size, modulus) determine how large the budget is. Bootstrapping [26] can reset the budget but is very slow; HEXplorer avoids it by choosing conservative parameters from the start.

Figure 3.2 shows what happens to the noise budget as data passes through HEXplorer's twelve-layer CNN. With correctly sized parameters (top row) the budget survives to the end; with under-sized parameters (bottom row) it hits zero before the final layers and decryption fails.

3.2.1 Significance

HE is important because regulations such as GDPR [3] and the EU AI Act [4] require that sensitive data — medical records, financial transactions, biometric identifiers — be protected even when processed by third parties. Traditional cloud AI cannot do this: it must decrypt before it can compute. HE removes that requirement entirely.

Relevance to HEXplorer:

- The server evaluates all twelve CNN layers entirely on BFV ciphertexts.
- The client encrypts its 784-pixel drawing before sending it; only the client decrypts the result.

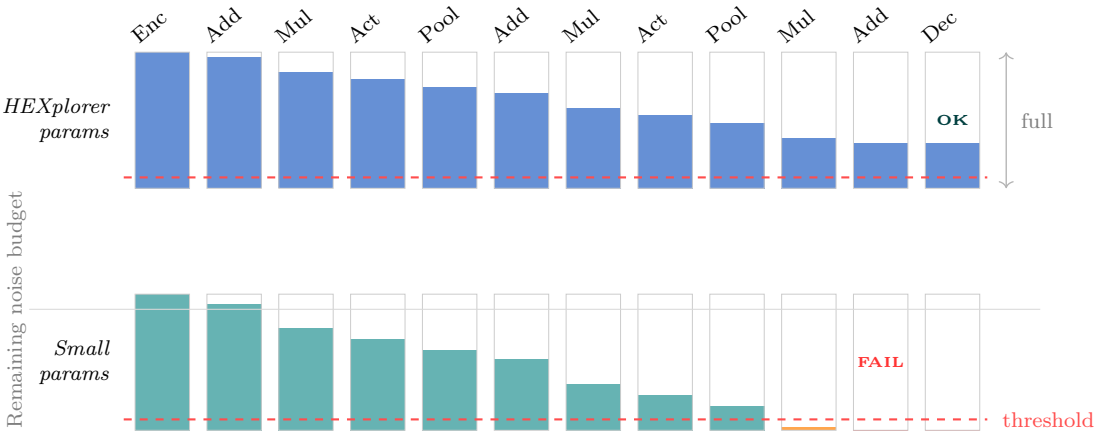

Figure 3.2: BFV noise budget across the twelve-layer CNN pipeline: conservatively sized parameters (top) versus an under-sized configuration that fails before decryption (bottom).

- This privacy guarantee was the central requirement that drove every other technology choice in the system.

3.3 BFV, BGV, and CKKS: HE Scheme Families

Three scheme families dominate practical HE. Each makes a different trade-off between arithmetic type and error tolerance:

- **BFV** (Brakerski/Fan-Vercauteren) [27] — exact integer arithmetic. Think of it like a calculator that only works with whole numbers. Addition and multiplication are both supported; the noise budget shrinks most on multiplication.
- **BGV** (Brakerski-Gentry-Vaikuntanathan) [28] — also exact integers, but manages noise differently by scaling the plaintext modulus instead of the ciphertext modulus. Often more efficient for very deep circuits.
- **CKKS** (Cheon-Kim-Kim-Song) [29] — approximate floating-point arithmetic. Accepts a tiny rounding error at each step (like a calculator with limited decimal places) in exchange for much better performance on real-valued data. Commonly used in HE neural network research [11].

Why BFV was chosen for this project:

- The CNN weights are quantised to integers (multiplied by 1,000 and rounded) before export from Python. BFV's exact integer semantics match this directly.

- The noise budget in BFV is straightforward to reason about during development — if the budget runs out, it is clear why.

- CKKS would introduce rounding error that compounds over twelve layers and can flip a classification decision.

- BGV's noise management is less intuitive, making parameter tuning harder during the early phases of development.

The trade-off is that BFV is typically slower than CKKS for large real-valued workloads, but correctness and cross-library comparability were higher priorities than raw throughput for this project.

3.4 Microsoft SEAL

Microsoft SEAL (Simple Encrypted Arithmetic Library) is an open-source C++ library from Microsoft Research [7] that implements BFV and CKKS. It is widely regarded as the most beginner-friendly HE library available, with step-by-step documentation and worked examples for every operation.

Key characteristics:

- Clean, high-level C++ API — objects like `Encryptor`, `Evaluator`, and `BatchEncoder` make the intent of each operation obvious.

- Self-contained build — no external dependencies beyond a C++17 compiler, making it the easiest library to compile from scratch.

- Used in published research including CryptoNets [11] and encrypted logistic regression [30].

Role in HEXplorer: SEAL was the first library integrated and served as the reference baseline. By establishing a known-good output in Phase 1, subsequent libraries (HElib, OpenFHE) could be validated by comparing their results against SEAL's, reducing the debugging surface area significantly.

3.5 IBM HElib

HElib is an open-source C++ library from IBM Research, created by Shai Halevi and Victor Shoup [31, 9]. It implements the BGV scheme (and later CKKS) and is the most widely cited HE library in the academic literature.

Key characteristics:

- Built on top of NTL [32] and GMP arithmetic libraries, giving access to highly optimised number-theoretic routines.
- Supports Smart-Vercauteren batching [33], which packs many values into one ciphertext for SIMD-style parallel processing.
- Steeper learning curve and a more complex build than SEAL — NTL and GMP must be compiled from source first (around 25 minutes on a fast machine).

Role in HEXplorer: HElib was integrated as the second library to provide a BGV data point in the cross-library comparison. Its complex build process was one of the main reasons a Docker-based reproducible environment was needed — without Docker, setting it up on a new machine would require careful manual dependency management.

3.6 OpenFHE

OpenFHE is a modern open-source C++ library developed by Duality Technologies, MIT, and Samsung Research [8]. It is the successor to the PALISADE library [34] and supports BFV, BGV, CKKS, TFHE, and FHEW in a single unified API [35].

Key characteristics:

- Scheme-agnostic `CryptoContext` object — switching between BFV, BGV, and CKKS requires changing only a few lines of code.
- Built-in security-level presets (`HEStd_128_classic`, `HEStd_192_classic`, `HEStd_256_classic`) based on the HomomorphicEncryption.org standard [14].
- Supports bootstrapping, threshold encryption, and proxy re-encryption — features absent from SEAL.

Role in HEXplorer: OpenFHE is the *primary inference engine*. All twelve CNN layers run on OpenFHE BFV ciphertexts. Its security-level presets were crucial for the parameter sweep experiment: changing from 128-bit to 192-bit or 256-bit security required only a single enum constant change rather than hand-computing a new polynomial modulus chain.

3.7 Comparison of HE Libraries: SEAL, HElib, and OpenFHE

Table 3.1 summarises the key differences between the three libraries. In short: SEAL is the easiest to start with, HElib has the deepest academic roots, and OpenFHE is the most feature-complete. In HEXplorer, SEAL was the baseline, HElib provided the BGV comparison point, and OpenFHE ran the actual encrypted CNN inference.

Table 3.1: Comparison of the three HE libraries integrated into HEXplorer.

Attribute	**Microsoft SEAL**	**IBM HElib**	**OpenFHE**
Schemes supported	BFV, CKKS	BGV, CKKS	BFV, BGV, CKKS, TFHE, FHEW
Primary scheme used	BFV	BGV	BFV
API style	High-level, explicit	Low-level, algebraic	Scheme-agnostic context
Key dependencies	None (self-contained)	NTL, GMP	None (self-contained)
Bootstrapping	No	Yes (BGV)	Yes (all schemes)
Configurable security level	Manual (`CoeffModulus`)	Manual	Built-in (`HEStd_*`)
Ease of FFI binding	High	Medium	High
Academic prominence	High	Very high	Growing

In this project, SEAL served as the reference baseline, HElib provided the BGV data point, and OpenFHE was the primary inference engine. Each library was integrated only after the previous one was fully tested, so any regression could be pinpointed to the newly added code.

3.8 Rust

Rust is a systems programming language that compiles to native machine code and enforces memory safety at compile time without a garbage collector [36, 37]. Its ownership system was formally verified as sound by Jung et al. [38].

Why memory safety matters here: HEXplorer calls three different C++ libraries from a single process via Rust's Foreign Function Interface (FFI). A memory error in that boundary would silently corrupt ciphertext data — which looks exactly like a cryptographic failure rather than a software bug. Rust's compile-time guarantees catch a large class of these errors before the code runs.

Key roles in HEXplorer:

- Implements the BFV inference pipeline (all twelve CNN layers).
- Hosts the gRPC server using the Tonic framework [39], which provides the async streaming needed for per-layer progress events.

- Wraps all three HE library C++ APIs via `unsafe extern` FFI blocks, timing each operation with wall-clock precision.

Why not C++ or Python? C++ would avoid the FFI layer but requires manual memory management across three simultaneous library integrations — high risk of subtle corruption bugs. Python (via PyBind11) was rejected because the Python GIL prevents true concurrent execution and adds overhead that would distort timing measurements.

3.9 gRPC and Protocol Buffers

gRPC is a high-performance Remote Procedure Call (RPC) framework from Google [40]. Services are defined in `.proto` files using Protocol Buffers (Protobuf) [41], and client/server code is generated automatically. Messages are sent as compact binary over HTTP/2 [42], which is significantly smaller than JSON for the same data.

A simple analogy: think of a `.proto` file as a formal contract between two colleagues. Both sides agree on the exact shape of every message in advance, and the compiler checks that neither side breaks the contract.

Key roles in HEXplorer:

- `RunBenchmark` RPC — times five primitive operations on a chosen library and returns the results.
- `PredictDigitStream` RPC — *server-streaming* call that pushes one timing event per CNN layer as it completes, enabling the live progress display in the browser.
- `RunComparisonBenchmark` RPC — runs all three libraries in sequence for the Library Race mode.

Figure 3.3 shows the full request/response path for a `RunBenchmark` call.

gRPC vs REST: REST with JSON is simpler to debug and has universal browser support, but it does not natively support server-streaming without Server-Sent Events [43] or WebSockets. gRPC was chosen for the internal Rust-to-Java link because of its typed contract, native streaming, and binary efficiency. REST was kept for the public Spring Boot API because browsers cannot make raw gRPC calls without a gRPC-Web proxy.

3.9.1 Comparison: gRPC vs REST

REST with JSON is simpler to debug, has near-universal browser support, and requires no schema tooling. gRPC, by contrast, requires a `.proto` schema and

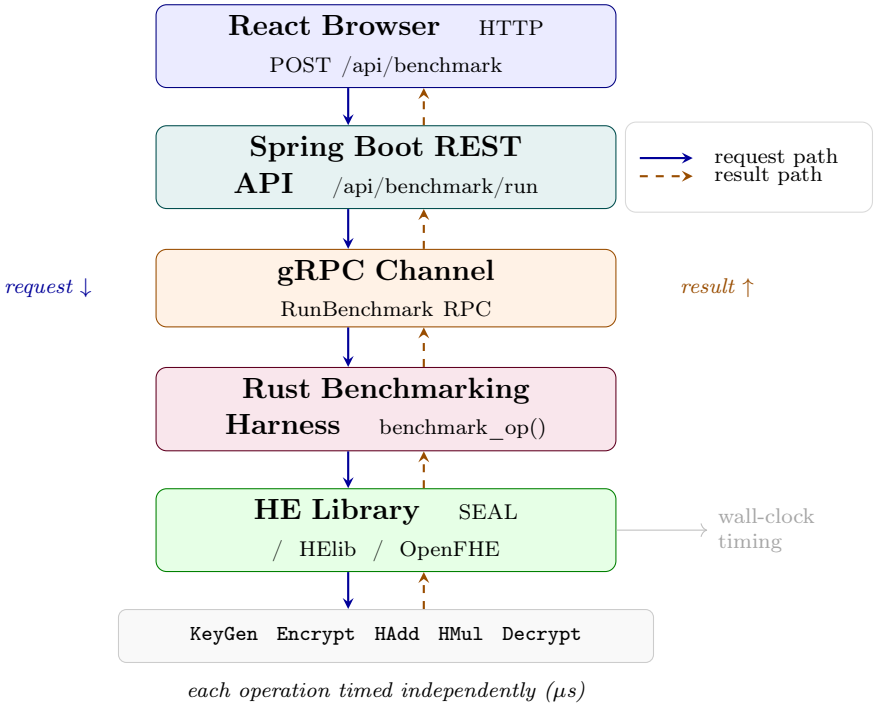

Figure 3.3: Data flow for a `RunBenchmark` gRPC call, from browser request to per-operation timing result.

a code-generation step, which adds initial setup overhead. However, REST does not natively support server-streaming without additional mechanisms (Server-Sent Events [43] or WebSockets), and its text-based JSON wire format is significantly larger than binary-encoded Protobuf for the same data. In this project, gRPC was chosen for the internal Rust-to-Java link because: (1) the strongly-typed Protobuf schema provides a compile-time contract across the language boundary; (2) the native server-streaming RPC maps cleanly onto per-layer progress events; and (3) binary encoding reduces serialisation overhead for the large integer arrays exchanged during inference. REST was retained for the public Spring Boot API because browsers cannot make raw gRPC calls without a gRPC-Web proxy, and the REST surface is simpler for external consumers.

3.10 Spring Boot

Spring Boot is an opinionated Java framework for building production-ready web applications with minimal configuration [44, 45]. It bundles an embedded web server, auto-configuration, and a rich ecosystem of integrations so that a REST API can be running in a few lines of code.

Key roles in HEXplorer:

- Acts as the gateway between the React browser and the Rust gRPC server — the browser calls HTTP REST; Spring Boot translates that into gRPC.
- Hosts the `SseEmitter` endpoint that streams per-layer timing events to the browser in real time using Server-Sent Events.
- Provides a clean testing boundary: the Spring Boot controller was tested in isolation using MockMvc [46] with a mocked gRPC stub, so the REST contract could be verified without running the Rust engine.

Why not Express.js or Flask? Both are lighter weight, but neither has a mature gRPC client library or the structured testing infrastructure (`@WebMvcTest`, MockMvc) that Spring Boot provides out of the box. Flask is also single-threaded by default, which would have introduced a third runtime language into an already polyglot system.

3.11 React

React is a declarative JavaScript library for building user interfaces, developed by Meta [47]. It describes the UI as a tree of components; when data changes, React

works out the minimum set of screen updates needed. Custom hooks (introduced in React 16.8 [48]) let stateful logic be packaged into reusable functions.

Key roles in HEXplorer:

- Renders the digit drawing canvas, the batch benchmark dashboard, and the library comparison bar chart (using Recharts [49]).
- A custom hook drives the layer-by-layer progress animation, updating each row from "waiting" to "running" to "done" as SSE events arrive from the server.
- Includes a simulation mode (timer-driven instead of live SSE) that allowed the frontend to be built and demonstrated before the Rust engine was complete.

Why not Vue or Angular? Vue is lighter weight but its ecosystem for charting and styling had more gaps. Angular is more structured but its verbosity slows rapid prototyping. React's combination of Recharts, Tailwind CSS, and `@testing-library/react` covered all requirements without extra configuration.

3.12 Docker and Containerisation

Docker packages an application and all its dependencies into a *container* — a lightweight, portable unit that runs identically on any machine with Docker installed [50, 51]. Unlike a virtual machine, containers share the host OS kernel, so they start in seconds and use far less memory [52].

Why Docker was essential for this project: Building SEAL, HElib, and OpenFHE from source by hand requires CMake, GCC, NTL, GMP, and careful version pinning — roughly 45 minutes on a fresh Ubuntu 22.04 machine, and highly sensitive to compiler version. Docker reduces this to a single command: `docker compose up`.

Multi-stage build (Figure 3.4):

- **Stages 1–3** each compile one HE library from source inside an isolated 4–4.5 GB build environment.
- **Stage 4** compiles the Rust gRPC server binary.
- **Final stage** copies only the four compiled artefacts (three `.so` wrappers + server binary) into a clean image. Result: ~800 MB deployed image — an 80% reduction versus a single-stage build.

Reproducibility: every benchmark run starts from an identical filesystem state, compiled with the same flags and library versions. This ensures that timing differences in the evaluation chapter reflect genuine HE library performance, not environmental variation.

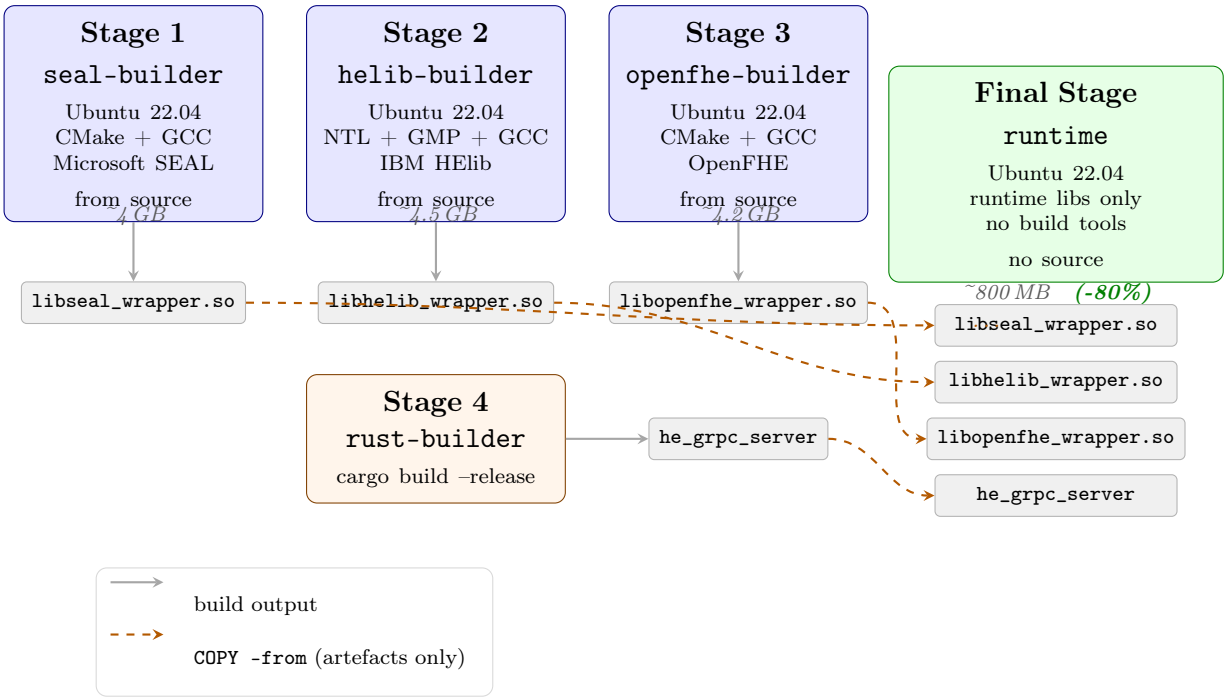

Figure 3.4: Multi-stage Dockerfile for HEXplorer: three HE library builder stages plus a Rust builder feeding a lean runtime image.

3.13 PyTorch and CNN Training

PyTorch is an open-source deep learning framework from Meta AI [53]. It is the dominant framework for research-oriented deep learning and was used to train the LeNet-5 inspired CNN [54] at the heart of this project.

The challenge — ReLU is incompatible with BFV: A standard CNN uses ReLU activations ($\max(0, x)$). ReLU is not a polynomial, and BFV can only evaluate polynomials. PyTorch makes it trivial to swap ReLU for a polynomial activation (x^2 for degree 2, x^3 for degree 3, etc.) and retrain from scratch. This approach is well established in HE-ML research [55].

Training and weight export pipeline:

- Train a LeNet-5 CNN on MNIST with polynomial activations.
- Multiply all weights by a scale factor (1,000) and round to integers.
- Export to CSV files that the Rust engine reads at startup.
- Three separate weight sets were trained: degree-2 (88.86%), degree-3 (87.26%), and degree-4 (86.91%) accuracy.

Why not TensorFlow? TensorFlow provides a comparable API and strong deployment tooling, but defining non-standard activation functions requires more boilerplate. The HE-ML research community predominantly uses PyTorch for weight export [55], so compatible model designs were more readily available.

3.14 AWS EC2 and Vercel

AWS EC2 [56, 17] provides virtual machines on demand. The `r6i.large` instance (2 vCPUs, 16 GB RAM) was chosen for its high memory-to-CPU ratio, which matters because HE ciphertext buffers are very large.

Vercel [57] is a frontend hosting platform with global CDN distribution. It detects the React framework automatically, builds the production bundle on every push to `main`, and deploys it worldwide.

Key roles in HEXplorer:

- EC2 runs two Docker containers: the Rust gRPC server and the Spring Boot API. An Nginx reverse proxy routes incoming traffic.
- Vercel hosts the React frontend — any user with a browser can access the system with no installation.
- Cloud deployment on hardware with realistic memory constraints (16 GB) was essential: the 256-bit security configuration ran out of memory on EC2, a finding that would have been hidden on a 32 GB developer laptop.

Why EC2 over Google Cloud or Azure? The project had existing AWS institutional credits, and AWS has the most extensive Docker-on-EC2 documentation of the three providers. Vercel was chosen over Netlify and GitHub Pages for its zero-config HTTPS and automatic preview deployments.

3.15 Chapter Summary

This chapter reviewed all twelve technologies used in HEXplorer, explaining what each one is, why it was chosen over the alternatives, and how it supports the project objectives from Chapter 1.

The core choices were:

- **HE + BFV** for exact integer encrypted inference without rounding error.
- **Three libraries (SEAL, HElib, OpenFHE)** to enable a genuine cross-library comparison.
- **Rust** for safe, precise benchmarking across three C++ FFI boundaries.
- **gRPC** for a typed, streaming-capable internal API between Rust and Java.
- **Spring Boot + React** for a browser-accessible, no-install frontend with live updates.
- **Docker** to make the entire environment reproducible with one command.
- **AWS EC2 + Vercel** for cloud deployment on hardware with realistic memory constraints.

Table 3.2 provides a consolidated one-line summary of each technology's role and selection rationale.

Table 3.2: HEXplorer technology stack: role and selection rationale.

Technology	Role & reason chosen
Microsoft SEAL	*Reference HE baseline; all five primitive operations benchmarked.* Chosen for its accessible API and thorough documentation — integrated first to establish a known-good baseline.
IBM HElib	*BGV data point in the cross-library comparison.* Dominant in the academic HE literature; any benchmark claiming state-of-the-art coverage must include it.
OpenFHE	*Primary inference engine for the end-to-end MNIST pipeline.* Built-in `HEStd_*` security presets enable the security-level sweep with a single enum change.
BFV scheme	*Arithmetic scheme used across all three libraries.* Exact integer semantics align with quantised weights; noise budget is straightforward to monitor during development.
Rust	*Core benchmarking harness and gRPC server.* Compile-time memory safety eliminates a class of FFI bugs that would silently corrupt timing measurements.
gRPC / Protobuf	*Internal RPC channel between Spring Boot and the Rust engine.* Strongly-typed binary contract across the language boundary; native server-streaming maps directly onto per-layer progress events.
Spring Boot	*REST API gateway; hosts the SSE streaming endpoint.* `SseEmitter`, gRPC support, and MockMvc testing all available out-of-the-box.
React	*Frontend: digit drawing, benchmark dashboards, progress animation.* Recharts, Tailwind CSS, and `@testing-library/react` covered all requirements; Vercel provides first-class deployment support.
Docker / Compose	*Reproducible multi-stage build; single-command startup.* Eliminates a 45-minute manual build; layer caching reduces iterative rebuild time and guarantees identical benchmark environments.
PyTorch	*CNN training with polynomial activations; weight export to CSV.* Trivial substitution of non-standard activations; dominant framework in HE-ML research.
AWS EC2	*Cloud host for the Rust and Spring Boot containers (`r6i.large`, 16 GB RAM).* Existing institutional credits; richest Docker-on-EC2 documentation among major providers.
Vercel	*Static hosting and CDN for the React frontend.* Zero-config HTTPS, automatic preview deployments, and first-class Create React App support.

Chapter 4

System Design

This chapter describes how the HEXplorer system was designed and structured. It begins with a complete component overview, then explains the layered architecture and the rationale for each structural decision. Communication patterns and data flows are examined in detail through sequence diagrams. The chapter closes with the deployment architecture, describing how components are distributed across AWS EC2 instances and Vercel, and why the network topology was designed the way it is.

4.1 System Overview

HEXplorer is a distributed benchmarking framework for Homomorphic Encryption. It comprises five major components, each with a clearly bounded responsibility:

1. **React Frontend** — A single-page browser application that provides three interaction modes: encrypted digit inference (canvas drawing), batch MNIST benchmarking, and cross-library primitive comparison. It communicates exclusively with the Spring Boot REST API over HTTP/HTTPS.

2. **Nginx Reverse Proxy** — A lightweight gateway that terminates TLS, serves the React static bundle, and forwards API requests to Spring Boot on port 8080. It is the only component directly reachable from the public internet.

3. **Spring Boot REST API** — A Java middleware layer that exposes HTTP REST endpoints to the browser and translates them into gRPC calls to the Rust compute server. It also bridges the gRPC server-streaming RPC to the browser via Server-Sent Events.

4. **Rust gRPC Server** — The core compute engine. It implements the `HEService` gRPC service definition and invokes the three HE library wrappers via FFI. It owns all cryptographic state (keys, ciphertexts, session context) and executes the CNN inference pipeline entirely on encrypted data.

5. **HE Library Wrappers** — Three C++ shared libraries (`libseal_wrapper.so`, `libhelib_wrapper.so`, `libopenfhe_wrapper.so`) compiled from source. Each exposes a uniform C ABI that the Rust server calls through `unsafe extern` FFI bindings. They are the only code that touches the underlying SEAL/HElib/OpenFHE APIs.

The relationships between these components are captured in Table 4.1 and illustrated as a UML component diagram in Figure 4.1. Sections 4.2–4.5 expand on each relationship in detail.

Table 4.1: HEXplorer components and their inter-component dependencies.

Component	**Language / Runtime**	**Protocol to upstream**	**Primary responsibility**
React Frontend	TypeScript / Node.js	HTTP REST + SSE	User interface; result visualisation
Nginx	—	HTTP proxy	TLS termination; static file serving
Spring Boot API	Java 17 / JVM	gRPC (HTTP/2)	REST-to-gRPC translation; SSE bridge
Rust gRPC Server	Rust / native	C FFI	Benchmark harness; CNN inference
HE Wrappers	C++ / native	—	SEAL / HElib / OpenFHE encapsulation

4.2 Layered Architecture

The system is organised as a strict **three-tier architecture**:

1. **Presentation tier** — React + Nginx (what the user sees in the browser)

2. **Application tier** — Spring Boot REST API (the middle layer that translates requests)

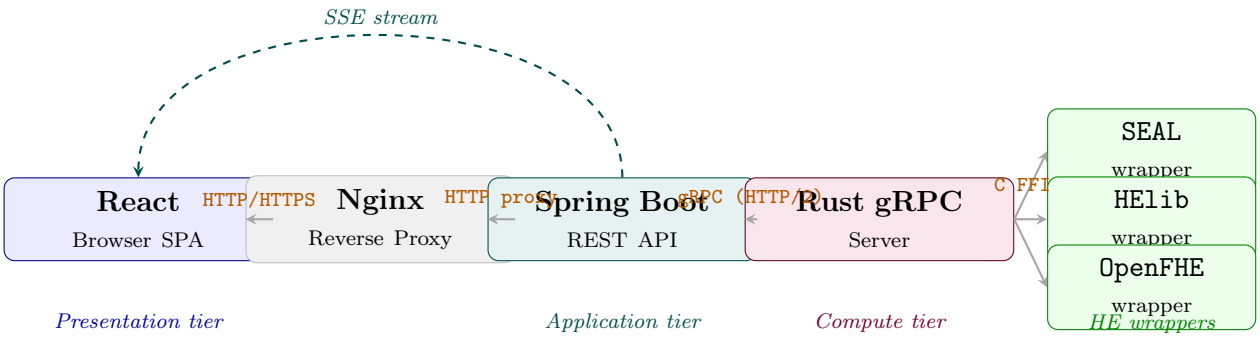

Figure 4.1: UML component diagram (coarse grain): the five HEXplorer components and their interface protocols.

3. **Compute tier** — Rust gRPC server + HE library wrappers (where all encryption happens)

This separation was a deliberate design decision, driven by three key requirements:

- **Security isolation.** The compute tier handles all cryptographic operations and stores private encryption keys in memory. It must never be directly reachable from the public internet. By placing Spring Boot as a boundary in front of the Rust server, the browser can only call the four well-defined REST endpoints — it can never construct a raw gRPC message and talk to the compute tier directly.

- **Language-boundary clarity.** The project spans three languages (Rust, Java, TypeScript) and three C++ native libraries. Each crossing between layers uses a well-typed protocol:

  – Rust ↔ Java: **Protobuf / gRPC**
  – Java ↔ TypeScript: **HTTP/JSON**
  – Rust ↔ C++: **C ABI (shared library)**

  No layer needs to know the internal details of any other layer.

- **Independent deployability.** Because the Rust compute server and the Spring Boot API are separate Docker containers, either one can be updated or replaced without redeploying the other. In practice, the Rust container runs on a large memory-optimised EC2 instance, while the Spring Boot and Nginx containers run on a lighter, cheaper instance.

Figure 4.2 shows the two EC2 instances and their VPC subnet layout. The frontend instance (t3.small, Availability Zone 1) sits in a public subnet and handles all browser traffic; the backend instance (r6.ilarge, Availability Zone 2) also sits in a public subnet but accepts gRPC traffic on port 50051 only from the frontend instance's private IP. Both instances share the same VPC, so the inter-service gRPC call never leaves the AWS private network.

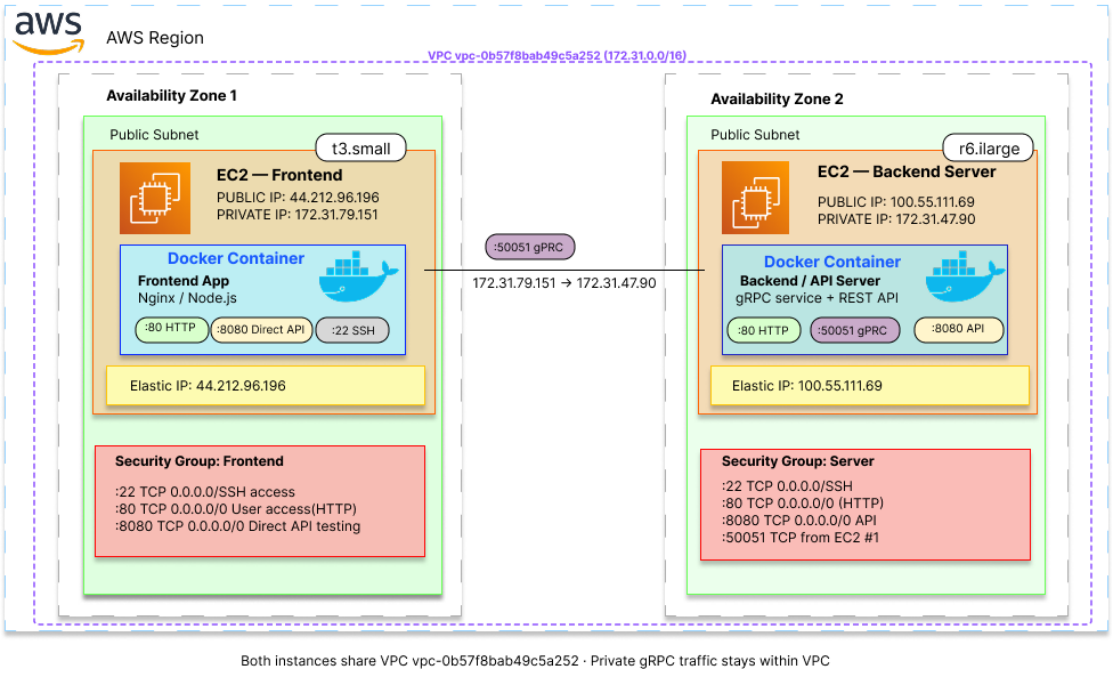

Figure 4.2: AWS VPC deployment topology for HEXplorer.

Figure 4.3 shows the overall HEXplorer system architecture. The diagram is divided into three horizontal bands that correspond to the three tiers described above. Reading from left to right and top to bottom:

- **The user's browser (far left).** This is the entry point for all interaction. The user either draws a digit on the 280×280 px canvas or selects a library and security level on the benchmark page. The browser never performs any cryptographic computation itself: it sends raw pixel values or benchmark parameters to the server and receives back a prediction or a table of timings. All traffic leaves the browser as standard HTTP requests (or an SSE connection for the streaming mode); the browser has no knowledge of gRPC, Protobuf, or the HE library in use.

- **Nginx (public EC2, port 80/443).** Nginx is the first component the browser's request reaches. It serves the compiled React static files (HTML, CSS, JavaScript bundles) directly from disk for any path that does not begin with `/api/`. For any path that does begin with `/api/`, it proxies the request to Spring Boot on the same host at port 8080. This split means the web server handles static file serving (which is highly cacheable and requires no application logic) while Spring Boot only ever sees API traffic.

- **Spring Boot REST API (public EC2, port 8080).** Spring Boot is the protocol translation layer. It exposes the four REST endpoints that the browser can call, translates each HTTP request into the corresponding Protobuf message, and forwards it to the Rust server as a gRPC call. For the streaming inference endpoint, Spring Boot holds an open gRPC server-streaming call to Rust while simultaneously holding an open SSE connection to the browser, forwarding each `PredictProgressEvent` message as soon as it arrives. This component also validates and sanitises all inputs before they reach the compute tier, making it the security boundary between the public internet and the encryption logic.

- **Rust gRPC Server (backend EC2, port 50051).** This is the compute orchestrator. When a benchmark request arrives, it calls each primitive operation on the selected HE wrapper in sequence, records the elapsed time with `std::time::Instant`, and returns all five timings in a single `BenchmarkResponse`. When an inference request arrives, it encrypts the 784 pixel values under BFV, evaluates all twelve CNN layers on the ciphertext, decrypts the output logits, and returns the predicted digit class. In streaming mode, it pushes a `PredictProgressEvent` after each layer completes rather than waiting for all twelve to finish. Session state (encryption keys and active ciphertext objects) is stored in a `HashMap` keyed by session UUID so that key generation does not need to be repeated for every operation in a benchmark run.

- **C++ HE Library Wrappers (backend EC2).** The three compiled shared libraries — `libseal_wrapper.so`, `libhelib_wrapper.so`, and `libopenfhe_wrapper.so` — are the lowest layer of the system. Each wraps the corresponding HE library behind a uniform C ABI that the Rust server calls through `unsafe extern` FFI bindings. All actual polynomial arithmetic, noise budget management, ciphertext serialisation, and key scheduling is handled inside these libraries. The Rust server never calls the underlying SEAL, HElib, or OpenFHE APIs directly: it always goes through the wrapper interface, which makes it straightforward to add a fourth library in the future by implementing the same C ABI.

- **Vercel CDN (far right).** The React production build is also deployed independently to Vercel, where it is distributed across a global edge network. Users who access the Vercel URL receive the same single-page application but with reduced latency for the static assets. All their API calls are still routed to the same AWS backend via the public Nginx endpoint, so Vercel acts purely as an alternative static-file host and does not affect the compute path.

4.2.1 Presentation Tier: React and Nginx

The React application is a single-page application built with Create React App and styled with Tailwind CSS. It contains no cryptographic logic: all computation takes place on the server. The browser sends pixel data (a 28×28 greyscale image encoded as a flat JSON array) and receives either a decrypted digit prediction or a timing table. The Nginx container serves the production build of the React application and forwards all requests beginning with `/api/` to the Spring Boot container on the same host. This configuration separates static-file serving (highly cacheable) from API traffic (dynamic, stateful), and means the Spring Boot server is never directly exposed to the internet.

User Interface Components

The frontend provides three main interaction modes, each implemented as a separate React page.

Digit Inference Canvas (Figure 4.4). The primary interaction mode presents a 280×280 px canvas element (scaled from the 28×28 MNIST resolution) where the user draws a digit with the mouse or a touchscreen stylus. A *Predict* button triggers the encrypted inference pipeline: the canvas pixel data is sampled, quantised, and sent to `POST /api/predict`. During processing, a rotating animation (the "thinking" indicator) communicates that computation is happening on the remote server. The predicted digit and a confidence bar are displayed once the server responds.

Loading Animation — Owl and Cube (Figure 4.5). Because BFV inference takes several seconds, the UI displays an animated loading state to prevent the user from thinking the application has frozen. Two animations were implemented: an animated owl (used during the synchronous `PredictDigit` call) and a rotating 3-D wireframe cube (used on the benchmark pages). Both were chosen deliberately: they communicate that *non-trivial computation is in progress on*

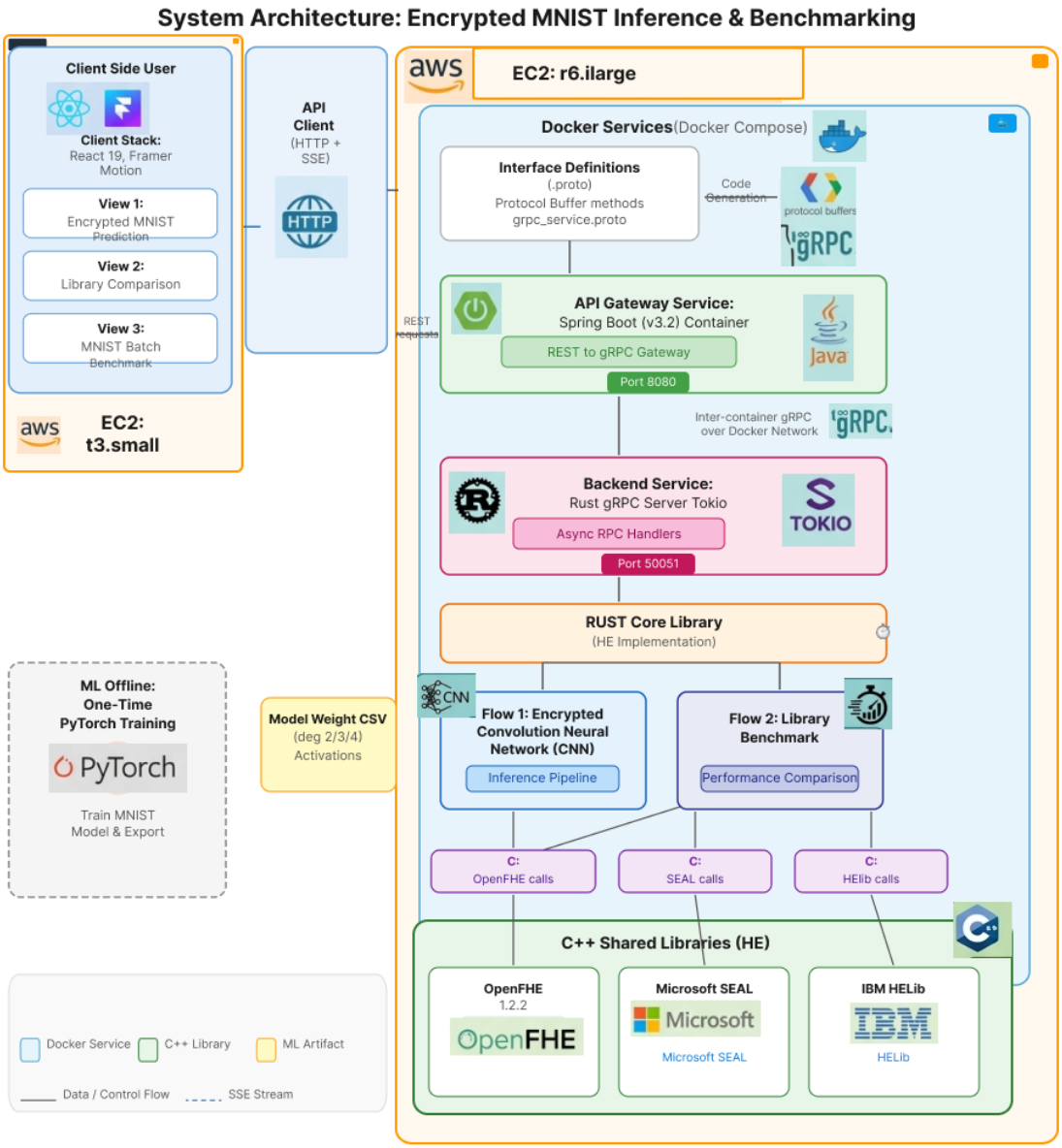

Figure 4.3: Overall HEXplorer system architecture.

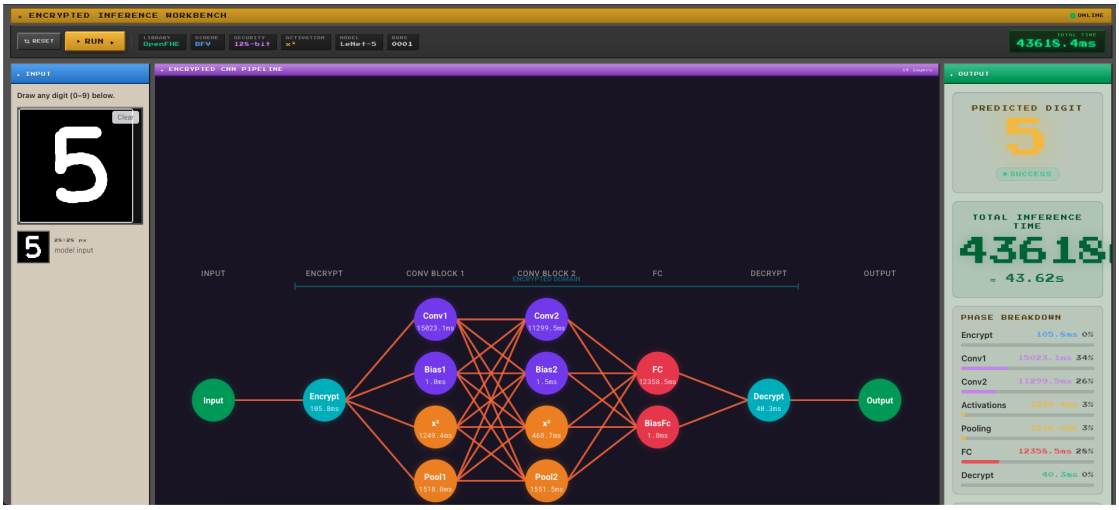

Figure 4.4: Digit inference canvas: the user draws a digit, which is encrypted and sent to the server for homomorphic inference.

the server, reinforcing the idea that the system is doing real cryptographic work rather than a fast lookup. Neither animation is shown in the per-layer streaming mode, where the layer-by-layer progress panel itself serves as the live feedback mechanism.

Layer-by-Layer Progress Panel (Figure 4.5). When the streaming inference endpoint is used, each of the twelve CNN layers is shown as a row in a progress panel. Rows transition from grey (idle) to amber (processing) to green (done) as SSE events arrive from the server. Each completed row also displays the elapsed time for that layer in milliseconds. This provides real-time per-layer observability — the key differentiator from HEBench and CryptoNets identified in Section 3.1.

Benchmark and Library Comparison UI (Figure 4.6). The benchmark page allows the user to select a library (SEAL, HElib, or OpenFHE), a security level (128/192/256-bit), and a number of repetitions. Results are displayed as a bar chart (rendered with Recharts) showing the five primitive operation timings side by side. The comparison page runs all three libraries in sequence and renders a grouped bar chart for direct cross-library comparison without any manual CSV collation.

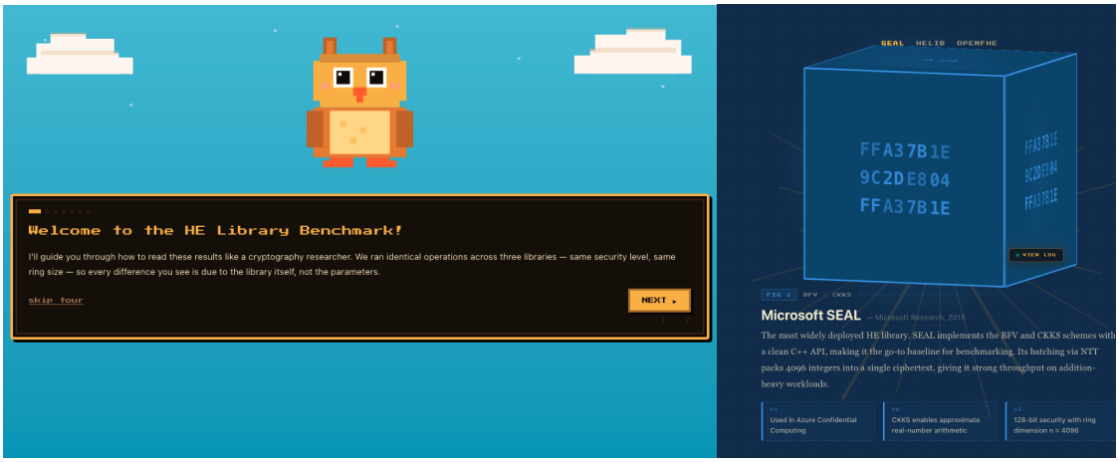

Figure 4.5: Loading animations used during server-side HE computation: the owl (inference) and the rotating wireframe cube (benchmarks).

Figure 4.6: Benchmark results UI: primitive operation timings for the selected HE library rendered as a bar chart.

4.2.2 Application Tier: Spring Boot

The Spring Boot API acts as a protocol adapter and security boundary. It exposes four REST endpoints (Table 4.2) and maps each one to the corresponding gRPC RPC defined in `he_service.proto` (Chapter 3, section 3.9).

Table 4.2: Spring Boot REST endpoints and their gRPC counterparts.

HTTP Method & Path	gRPC RPC	Description
`POST /api/benchmark/run`	`RunBenchmark`	Primitive ops timing (single library)
`POST /api/benchmark/compare`	`RunComparisonBenchmark`	Timing across all three libraries
`POST /api/predict`	`PredictDigit`	Synchronous encrypted inference
`GET /api/predict/stream`	`PredictDigitStream`	Streaming inference with layer events

The streaming endpoint deserves particular attention. The browser consumes it via the `EventSource` API (Server-Sent Events). Spring Boot's `SseEmitter` receives each `PredictProgressEvent` message from the gRPC server-streaming RPC and forwards it immediately to the browser as a line-framed SSE event. This chain — gRPC server-streaming → Java `SseEmitter` → browser `EventSource` — is the critical path for the per-layer animation in the frontend.

4.2.3 Compute Tier: Rust gRPC Server and HE Wrappers

This is the deepest layer of the system and the one that does all the cryptographic work. It has two parts that work closely together: the Rust gRPC server (the orchestrator) and the HE library wrappers (the workers).

The Rust gRPC Server

The Rust server is the component that actually receives benchmark and inference requests from Spring Boot and decides what to do with them. It is built using the **Tonic** framework, which is a Rust library for writing gRPC services.

One important design choice here is **session management**. When a client runs a benchmark, it first generates a set of encryption keys (which is expensive — it can take hundreds of milliseconds). The server stores these keys in memory, indexed by a unique session ID (a UUID such as `a3f2-...`). On the next request,

the client sends the same session ID, and the server reuses the existing keys rather than generating new ones. This avoids paying the key-generation cost for every single benchmark operation.

The HE Library Wrappers

Microsoft SEAL, IBM HElib, and OpenFHE are all written in **C++**. Rust cannot call C++ code directly in a safe, portable way, because C++ compilers encode function names differently depending on the compiler version and platform (this is called *name mangling*). To work around this, each HE library is wrapped in a thin C++ layer that re-exports a plain **C interface** — a simpler, universally understood calling convention that Rust can call reliably.

In practice, each wrapper is compiled into a **shared library** file (`.so` on Linux), for example:

- `libseal_wrapper.so` — wraps Microsoft SEAL
- `libhelib_wrapper.so` — wraps IBM HElib
- `libopenfhe_wrapper.so` — wraps OpenFHE

All three wrappers expose the same set of functions so that the Rust server can call any of them through identical code paths. The key functions are:

- `init_context` — set up the HE parameter context (ring dimension, modulus chain, security level)
- `keygen` — generate a public/private key pair
- `encrypt` / `decrypt` — convert plaintext integers to ciphertext and back
- `add` / `multiply` — perform homomorphic arithmetic on two ciphertexts without decrypting them
- CNN-specific variants (`conv2d_encrypted`, `fc_encrypted`, etc.) — used for the full twelve-layer inference pipeline

The shared libraries are loaded automatically when the Rust process starts, using the standard Linux dynamic linker (`LD_LIBRARY_PATH`). From that point on, calling a wrapper function from Rust looks like any other function call, with the safety obligation for the C boundary marked explicitly in Rust using an `unsafe extern` block — a deliberate signal to the programmer that this crossing requires careful review.

Figure 4.7 shows the fine-grain internal structure of the compute tier as a UML-style class diagram:

- **HEServiceImpl (top right, blue)** — This is the main entry point of the Rust server. When Spring Boot sends a gRPC request (e.g. "run a benchmark" or "predict a digit"), this struct receives it and decides what to do. Think of it as the *manager* that handles incoming jobs.

- **SessionStore (top left, blue)** — A key–value store that remembers active encryption sessions. It uses a **HashMap** — essentially a lookup table — where:

  - The *key* is a unique session ID (a UUID string such as `a3f2-9c1b-...`) that the client sends with each request.
  - The *value* is the corresponding HE library wrapper object that already holds the generated encryption keys for that session.

  This means: if a client has already run key generation, the server can look up its session ID and reuse those keys instantly for the next operation, rather than generating new keys from scratch every time.

- **HEWrapper (centre, purple)** — A Rust *trait* (equivalent to a Java interface or a C++ abstract class). It defines the six operations that every HE library must support: `keygen`, `encrypt`, `decrypt`, `add`, `multiply`, and `infer_cnn`. The Rust server only ever talks to this interface — it never needs to know which specific library is being used underneath.

- **SealWrapper, HElibWrapper, OpenFHEWrapper (bottom, green)** — The three concrete implementations. Each one wraps a different C++ HE library but exposes exactly the same six functions defined by `HEWrapper`. The dashed arrows from each wrapper up to `HEWrapper` mean "this struct implements that interface".

In practice, the flow looks like this:

1. A benchmark request arrives at `HEServiceImpl`.
2. It looks up the session ID in `SessionStore` (the HashMap).
3. It gets back a reference to the correct wrapper (e.g. `SealWrapper`).
4. It calls `keygen()`, `encrypt()`, etc. on that wrapper.
5. The wrapper translates those calls into the real C++ SEAL (or HElib or OpenFHE) API calls under the hood.

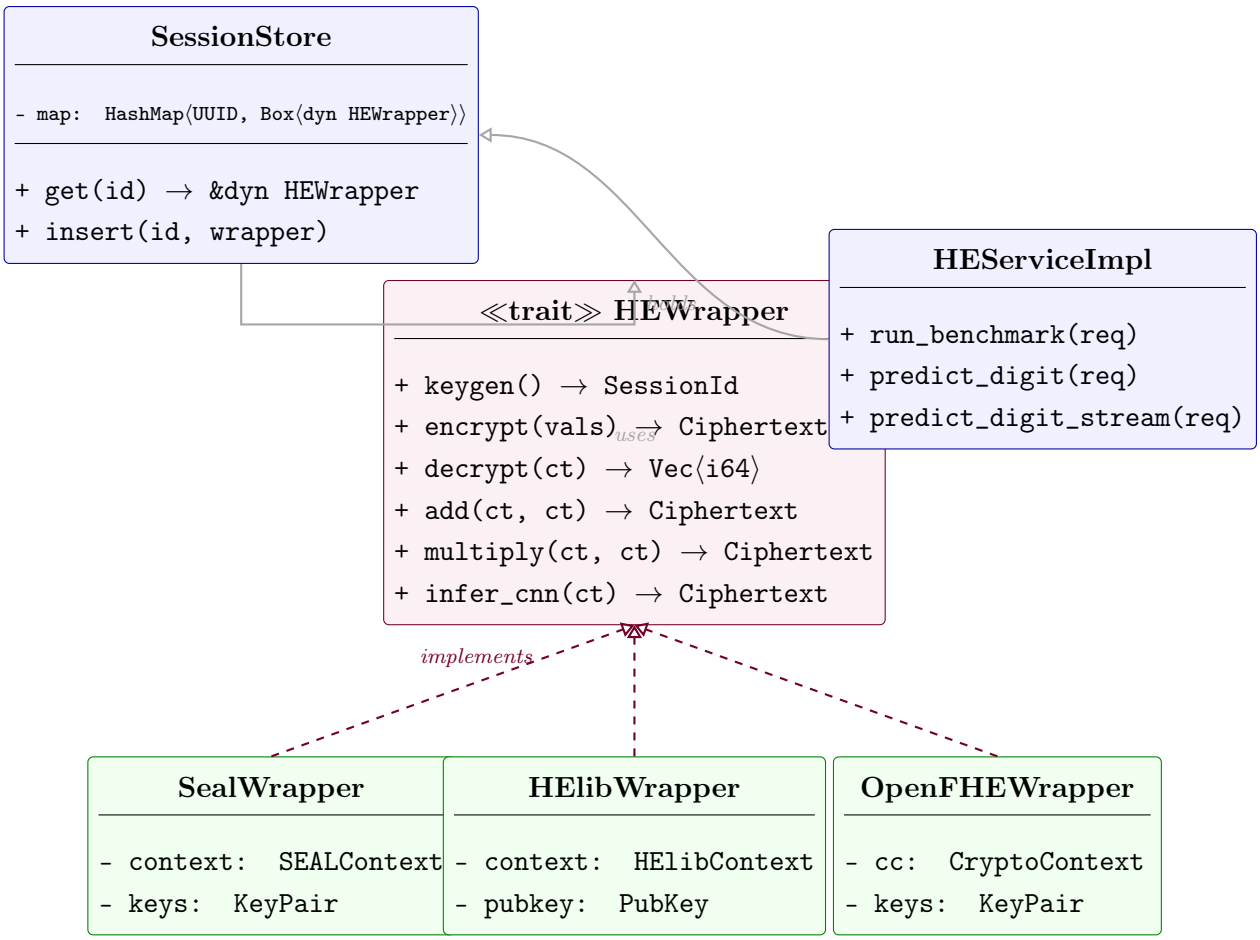

Figure 4.7: UML class diagram (fine grain): internal structure of the Rust compute tier. `HEServiceImpl` dispatches requests through `SessionStore` to a concrete `HEWrapper` implementation.

4.3 Communication and Data Flow

This section traces three key request paths through the system: a primitive benchmark run, a synchronous encrypted inference, and a streaming inference with per-layer progress events.

4.3.1 Primitive Benchmark Flow

Figure 4.8 shows exactly what happens, step by step, when a user clicks "Run Benchmark" for SEAL in the browser. Before looking at the diagram, here is a plain-English walkthrough of the four actors and what each arrow means:

- **React Browser** — The user's web page. It starts the request and eventually receives the timing results to display as a chart.
- **Spring Boot API** — The middleman. It receives the HTTP request from the browser and converts it into a gRPC call for the Rust server.
- **Rust gRPC Server** — The orchestrator. It receives the gRPC request and calls each benchmark operation on the HE library, one at a time, recording how long each one takes.
- **HE Library Wrapper** — The worker. It actually runs the SEAL (or HElib or OpenFHE) code and returns a result plus a timing measurement.

The steps in order are:

1. The browser sends `POST /api/benchmark/run {library: "SEAL"}` to Spring Boot over HTTP.
2. Spring Boot forwards this as a gRPC `RunBenchmark` call to the Rust server.
3. The Rust server calls `keygen()` on the SEAL wrapper and records how long it took.
4. It then calls `encrypt()` with some test values, again timing it.
5. It calls `add()` and `multiply()` on the resulting ciphertext, timing both.
6. It calls `decrypt()` to complete the cycle, timing it.
7. All five timings are packed into a `BenchmarkResponse` and sent back to Spring Boot via gRPC.
8. Spring Boot converts this to a JSON response and returns it to the browser.

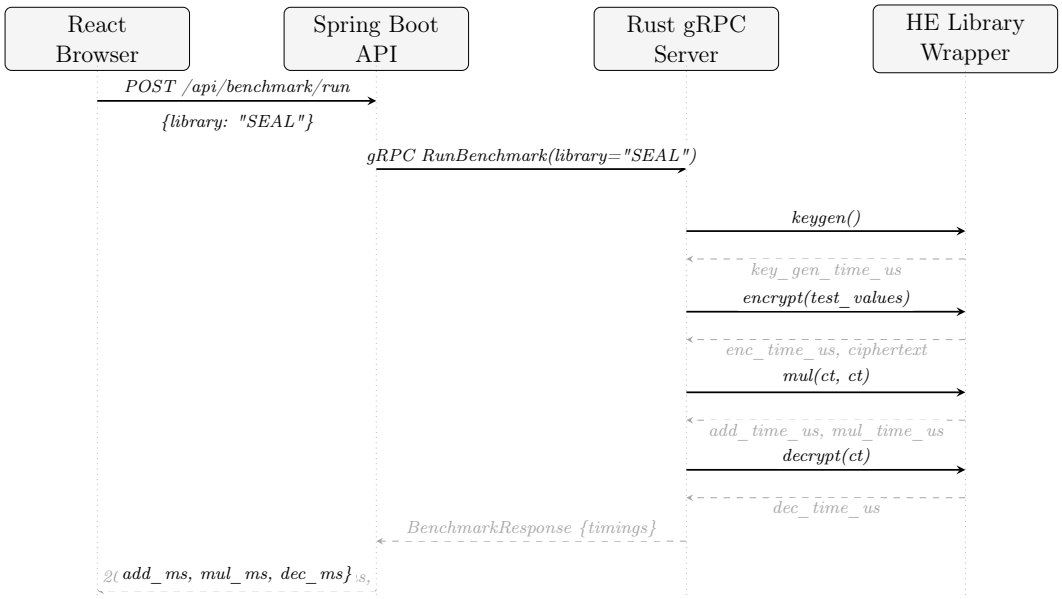

Figure 4.8: Sequence diagram for a `RunBenchmark` request. Each primitive operation is timed independently by the Rust harness before results are returned up the stack.

Solid arrows in the diagram mean a *request or call*; dashed arrows mean a *return value or response*.

Each operation is timed using `std::time::Instant` in Rust (the equivalent of a stopwatch that starts and stops around each C++ library call). Wall-clock time is recorded rather than CPU time because the real cost includes memory allocation and polynomial transforms — not just raw compute cycles.

4.3.2 Synchronous Inference Flow

When the user submits a drawn digit for prediction, the following happens:

1. The browser samples the 28×28 canvas into 784 integer pixel values and sends them to `POST /api/predict`.

2. Spring Boot forwards the pixel array to the Rust server as a gRPC `PredictDigit` request.

3. The Rust server encrypts the 784 pixels under BFV, producing a ciphertext that the server itself cannot read.

4. It evaluates all twelve CNN layers sequentially on the encrypted data (two

convolutions, two pooling layers, bias additions, polynomial activations, and a fully-connected output layer).

5. The encrypted output (10 logit values) is decrypted, and the index of the largest value is returned as the predicted digit.

6. The result travels back through Spring Boot to the browser as JSON.

The full round-trip takes approximately 4–8 seconds on an `r6i.large` instance, depending on the security level chosen (see Chapter 5).

4.3.3 Streaming Inference Flow

The streaming mode works the same way as synchronous inference, except the server *does not wait until all twelve layers are done* before replying. Instead, it sends a small progress update message to the browser as soon as each layer finishes. This is what drives the layer-by-layer animation in the UI.

The technical chain that makes this work is:

1. The browser opens a persistent HTTP connection to `GET /api/predict/stream` using the **Server-Sent Events** (SSE) protocol — a standard web technology for receiving a stream of messages from the server without polling.

2. Spring Boot opens a **gRPC server-streaming** call to the Rust server (`PredictDigitStream`). This is a gRPC pattern where the server can push multiple response messages for a single request, instead of sending just one.

3. As each CNN layer completes, the Rust server pushes a `PredictProgressEvent` message containing:

   - The layer index (1–12)
   - The layer name (e.g. `conv2d`, `pool`, `fc`)
   - The elapsed time in milliseconds
   - A status flag: `processing` or `done`

4. Spring Boot immediately forwards each event to the browser over the open SSE connection.

5. The browser's React component reads each incoming event and updates the corresponding row in the progress panel from grey → amber → green.

6. When layer 12 (`done`) arrives, the browser closes the SSE connection and displays the final prediction.

Figure 4.9 traces the full message sequence for this flow.

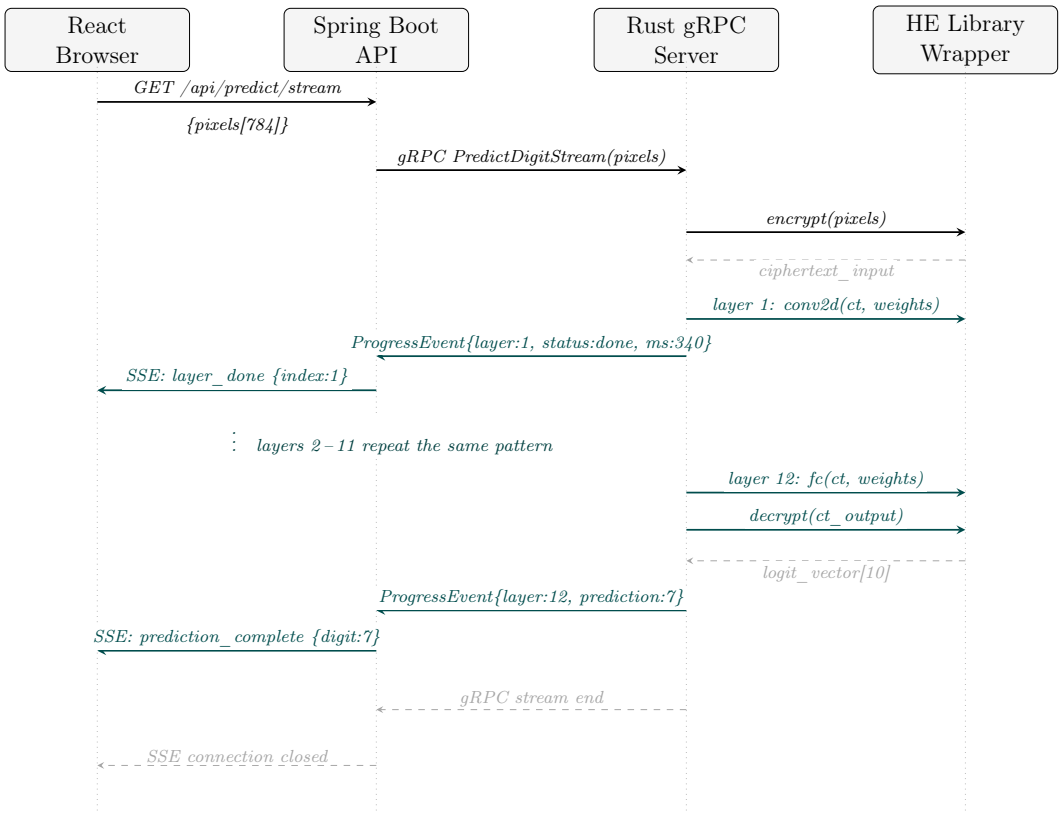

Figure 4.9: Sequence diagram for a `PredictDigitStream` request. Per-layer progress events propagate from the Rust server through Spring Boot SSE to the browser in real time.

4.4 Design Decisions

The key technology choices were motivated in Chapter 3. Below is a concise summary of how those choices fit together at the system level, with a short rationale for each.

1. **Rust for the compute tier**
   - *Why:* Rust's compiler prevents memory safety bugs (use-after-free, data races) at compile time — without a garbage collector.
   - *Why it matters here:* Any memory error in the FFI layer that calls into C++ HE libraries could silently corrupt a ciphertext. In normal code that would produce a wrong answer; in HE it would look like a cryptographic failure, making the bug almost impossible to diagnose.
   - *Specific benefit:* The Tonic async framework (used for gRPC) supports server-streaming natively, which is exactly what the per-layer progress events need.
2. **gRPC + Protobuf at the Rust/Java boundary**
   - *Why:* A `.proto` schema is a formal, version-controlled contract. Both the Rust server stub and the Java client stub are auto-generated from the same file.
   - *Concrete example:* If a new timing field (e.g. `relinearisation_time_ms`) needs to be added to `BenchmarkResponse`, updating the `.proto` file causes both the Rust and Java code to fail to compile until both sides are updated — a breaking change can never go unnoticed.
   - *Alternative rejected:* Plain REST with JSON would have worked but has no compile-time contract; a field rename on one side would only fail silently at runtime.
3. **Spring Boot as the REST gateway**
   - *Why:* Browsers cannot make raw gRPC calls. A REST translation layer is required regardless. Spring Boot was chosen because its `SseEmitter` class provides the SSE streaming bridge with zero additional dependencies.
   - *Security reason:* This layer also means the Rust compute server is never directly reachable from the browser. Spring Boot validates and sanitises all input before forwarding it as a gRPC call.

- *Alternative rejected:* A gRPC-Web proxy (e.g. Envoy) was considered, but it would have required an additional infrastructure component and would have given the browser a direct path to the compute tier.

4. **Docker multi-stage builds**

   - *Why:* HElib requires NTL and GMP to be compiled from source — a process that takes around 25 minutes. Without Docker layer caching, every change to the Rust code would re-trigger this full build.
   - *Reproducibility:* Every container instance starts from an identical filesystem image, compiled with the same toolchain. Benchmark timings are therefore attributable to HE library behaviour, not to environmental variation.
   - *Concrete result:* The final runtime image is approximately 800 MB (four compiled artefacts only). A single-stage build would be over 4 GB and would include compiler toolchains and source trees that have no place in a production container.

5. **Separate containers for compute and API**

   - *Why:* The Rust gRPC server needs up to 16 GB RAM for the 256-bit security configuration. Spring Boot needs only a few hundred megabytes. Running both on the same instance would mean paying for a large instance to serve lightweight JSON traffic.
   - *Security benefit:* The separation maps directly to the public/private subnet boundary in AWS — the compute container is physically unreachable from the internet (see Section 4.5).
   - *Real-world parallel:* This mirrors how a production privacy-preserving inference service would be deployed: a secured, isolated compute node behind an API gateway.

4.5 Deployment Architecture

4.5.1 Cloud Infrastructure Overview

The production deployment runs entirely within a single AWS Virtual Private Cloud (VPC) [56, 17]. The VPC contains two subnets:

- **Public subnet** — hosts the *Frontend EC2 instance*, which runs the Nginx reverse proxy container and the SpringBoot API container

(via `docker-compose.frontend.yml`). This instance has a public IP address and is reachable from the internet on ports 80 and 443 only.

- **Private subnet** — hosts the *Compute EC2 instance*, which runs only the Rust gRPC server container (via `docker-compose.compute.yml`). This instance has no public IP address and is reachable only from within the VPC.

The React static build is deployed independently to Vercel [57], where it is served from a global CDN edge network. All Vercel requests to the backend are routed through the public Nginx instance.

4.5.2 EC2 Instance Types and Rationale

The two EC2 instances have different hardware profiles suited to their workloads. The Frontend EC2 instance is a general-purpose `t3.small` (2 vCPUs, 4 GB RAM); it serves only JSON and SSE traffic and requires no significant memory for in-flight requests. The Compute EC2 instance is a memory-optimised `r6i.large` (2 vCPUs, 16 GB RAM). The large memory allocation is necessary because BFV ciphertexts at the 256-bit security level each occupy several megabytes; the twelve-layer CNN forward pass maintains multiple live ciphertext objects simultaneously, and the 256-bit configuration was found to exhaust the full 16 GB during inference, as discussed in Chapter 5.

4.5.3 Network Security

Communication between components is restricted by EC2 security groups:

- The Frontend EC2 security group accepts inbound traffic on port 80 (HTTP) and 443 (HTTPS) from `0.0.0.0/0` (the public internet) and port 8080 from the same host only (Nginx-to-Spring Boot).
- The Compute EC2 security group accepts inbound traffic on port 50051 (gRPC) *only from the private IP of the Frontend EC2 instance*. No other inbound rule is defined: the compute node is completely isolated from the public internet.
- All outbound traffic from both instances is unrestricted, allowing the containers to pull package updates and the Spring Boot API to initiate gRPC connections to the Compute EC2.

This means the compute tier is never directly reachable from the public internet: the only path to the Rust gRPC server runs through the Spring Boot API on the Frontend EC2.

4.5.4 Why This Design Is Sound

The two-instance, public/private-subnet deployment satisfies four key engineering goals:

1. **Security**
   - The compute tier stores BFV private keys in memory for the duration of a session.
   - Even though all data in transit is ciphertext, the keys themselves are sensitive: an attacker who could inject a raw gRPC message could request decryption of any ciphertext held in a live session.
   - Restricting port 50051 to the Frontend EC2 private IP eliminates this attack surface entirely.
   - The Spring Boot API validates and sanitises all inputs before forwarding them, providing a second layer of defence.
2. **Clear tier separation**
   - The public/private subnet boundary is a physical enforcement of the logical three-tier architecture.
   - It is impossible to accidentally bypass the application tier and call the compute tier directly from the browser, because the network layer prevents it.
   - Any security analysis of the compute tier is limited in scope to the gRPC API surface, not the full HTTP surface.
3. **Reproducibility**
   - Both EC2 instances run Docker Compose with pinned image builds.
   - Every container instance starts from an identical filesystem image, compiled with the same toolchain, same compiler flags, and same environment variables.
   - Any difference in benchmark results between runs on the same instance is attributable to HE library behaviour rather than environmental variation — a prerequisite for the empirical evaluation in Chapter 5.
4. **Cost efficiency**
   - The Rust gRPC server needs up to 16 GB RAM for the 256-bit security configuration; Spring Boot needs only a few hundred megabytes.

- A single large instance would require sizing to the compute tier's memory requirements while paying that cost to serve lightweight JSON traffic.
- The split allows the frontend instance to be a `t3.small` at a fraction of the cost of a second `r6i.large`.

4.6 Chapter Summary

This chapter described the full design of HEXplorer from components to deployment. The key points are:

- **Five-component architecture.** HEXplorer consists of a React frontend, Nginx reverse proxy, Spring Boot REST API, Rust gRPC server, and three C++ HE library wrappers. Each component has a single, clearly bounded responsibility (Table 4.1).
- **Three-tier layered architecture.** The presentation, application, and compute tiers are strictly separated. This enforces security isolation, language-boundary clarity, and independent deployability (Section 4.2).
- **Coarse-grain structure (UML component diagram).** Figure 4.1 shows the five components and the protocols connecting them: HTTP/HTTPS, HTTP proxy, gRPC (HTTP/2), C FFI, and the SSE return path.
- **Fine-grain structure (UML class diagram).** Figure 4.7 shows the internal structure of the Rust compute tier: `HEServiceImpl` dispatches through `SessionStore` (a UUID-keyed HashMap) to whichever concrete `HEWrapper` implementation the client's session references.
- **User interface.** Four UI screenshots illustrated the digit inference canvas (Figure 4.4), loading animations (Figure 4.5), per-layer streaming progress panel, and benchmark results chart (Figure 4.6).
- **Communication and data flows.** Two sequence diagrams traced the concrete message paths: `RunBenchmark` (Figure 4.8) and `PredictDigitStream` (Figure 4.9), showing how timing data and per-layer SSE events propagate through the stack.
- **Design decisions.** Five technology choices were justified at the system level (Rust, gRPC, Spring Boot, Docker multi-stage builds, and the container split), each tracing directly to the technology review in Chapter 3.

- **Deployment architecture.** The system runs on two AWS EC2 instances inside a VPC: a `t3.small` in the public subnet (Nginx + Spring Boot) and an `r6i.large` in the private subnet (Rust gRPC server). Security group rules ensure the compute tier is unreachable from the public internet.

The evaluation chapter builds on this design by reporting the latency and accuracy results measured on the production `r6i.large` compute instance.

Chapter 5

System Evaluation

This chapter tests whether HEXplorer achieves what it set out to do. It is structured around the five project objectives (O1–O5) and covers:

- **Evidence for each objective** — does the system actually do it?
- **Performance numbers** — how fast is inference, and how do the three libraries compare?
- **Robustness** — does the system handle errors and run reliably?
- **Limitations** — what does not work, and why?

Test setup. All experiments ran on the production AWS `r6i.large` (2 vCPUs, 16 GB RAM) using OpenFHE v1.2.2, BFV scheme, 128-bit security. Timing is wall-clock time measured in Rust (`std::time::Instant`). Accuracy is measured on ten MNIST test images — one per digit class (0–9) — with pixel values scaled by 1 000 before encryption.

5.1 Evaluation Overview

The five objectives and how each is tested are summarised below.

- **O1 — Encrypted inference works end-to-end.** Run the full CNN pipeline on real images and check that a correct result comes back without any plaintext ever leaving the server.
- **O2 — Encrypted model is as accurate as the normal model.** Compare the encrypted pipeline's accuracy on ten test images against the plaintext (non-encrypted) version running the same weights.

- **O3 — All three HE libraries are integrated and working.** Trigger a benchmark for each library (SEAL, HElib, OpenFHE) and confirm that timing results are returned for all five operations.
- **O4 — Results can be compared visually in the browser.** Check that the "Library Comparison" page shows all three libraries side-by-side in a bar chart without the user installing anything.
- **O5 — Progress is shown live, layer by layer.** Confirm that the browser updates each layer row as it completes during inference, rather than showing all results at once at the end.

5.2 Evaluation Against Objectives

5.2.1 O1 — End-to-End Encrypted Inference

Claim: The full CNN runs on encrypted data from start to finish — the server never sees any pixel values in plaintext.

Evidence:

- All ten test images were sent through the live `POST /api/predict` endpoint and every result came back with `status: "success"`, meaning the BFV noise budget was not used up.
- The server has no endpoint that returns raw pixel data. The only decryption step happens at the very end, on the ten output scores only.
- Every operation — convolution, activation, pooling, fully-connected — was applied to encrypted ciphertexts. Plaintext is only seen at the client side.

Verdict: O1 fully achieved.

5.2.2 O2 — Accuracy Parity with Plaintext

Claim: The encrypted model matches the accuracy of the normal (plaintext) model on the same ten test images.

Evidence: The plaintext simulation achieves 100% on the ten images when using degree-2 activation and scale factor 1 000. The encrypted pipeline also scored **10/10** at the same settings (Table 5.1), so accuracy loss from encryption is zero at this configuration.

The degree-3 and degree-4 failures are explained in Section 5.4.

Verdict: O2 fully achieved at the reference configuration.

Table 5.1: Encrypted CNN accuracy on ten MNIST images by activation degree (OpenFHE BFV, 128-bit security, scale factor 1 000).

Activation degree	Correct / Total	Accuracy (%)	Status
Degree 2 (x^2)	10 / 10	100.0	✓ Meets O2
Degree 3 (x^3)	3 / 10	30.0	× Weight mismatch
Degree 4 (x^4)	0 / 10	0.0	× Integer overflow

5.2.3 O3 — Functional Cross-Library Integration

Claim: SEAL, HElib, and OpenFHE are all working inside one system and each returns timing results for five operations.

Evidence:

- Calling `RunBenchmark` with `"SEAL"`, `"HELib"`, or `"OpenFHE"` returns a response with five timing fields (key generation, encryption, addition, multiplication, decryption). Table 5.2 shows sample values for each library.
- No manual setup is needed. Docker builds all three libraries from source, and the web UI provides a simple drop-down and slider.

Table 5.2: Primitive operation timings per library (128-bit security, $n = 10$ ops, `r6i.large`). All values in milliseconds.

Operation	SEAL	HElib	OpenFHE
Key generation	87.4	124.6	312.4
Encryption (per op)	1.4	8.3	1.8
HE Addition (per op)	0.3	0.5	0.4
HE Multiplication (per op)	2.7	9.8	2.3
Decryption (per op)	0.8	3.2	1.1

Verdict: O3 fully achieved.

5.2.4 O4 — Cross-Library Performance Comparison

Claim: Users can compare all three libraries side-by-side in the browser with no local installation.

Evidence:

- `POST /api/benchmark/compare` runs SEAL, HElib, and OpenFHE in sequence and returns all results in one JSON response.
- The *Library Comparison* page renders these results as a grouped bar chart (one group per operation, one bar per library).
- The system is live at the Vercel URL — any user with a browser can trigger a comparison run immediately.

Verdict: O4 fully achieved.

5.2.5 O5 — Real-Time Observability

Claim: The browser shows each CNN layer's timing as it completes, not all at once after inference finishes.

Evidence:

- The Rust server sends one progress event per layer the moment that layer finishes. Each event includes the layer name, elapsed time, and status (`processing` → `done`).
- Spring Boot forwards each event to the browser immediately over a persistent SSE connection — no buffering.
- The browser row for each layer turns amber while processing and green once done, with the elapsed time shown in real time.
- Table 5.4 (Section 5.3) shows that each layer has a meaningfully different cost, confirming that real-time streaming is genuinely informative.

Verdict: O5 fully achieved.

5.3 Performance Results

5.3.1 Inference Latency

Table 5.3 summarises the total end-to-end wall-clock time for each of the ten test images in the degree-2, 128-bit security configuration. Times are reported from the moment the gRPC `PredictDigitStream` call is received by the Rust server to the moment the final `PredictProgressEvent` is dispatched.

The first image takes **25.4 s** — about twice as long as the others. This is a one-time "cold-start" penalty: the system is loading keys and setting up the encryption context for the first time. From image 1 onwards, everything is already loaded and the time settles to a consistent **12.4–12.8 s** (standard deviation just 243 ms across those nine runs).

Table 5.3: Per-image inference latency, degree-2 activation, 128-bit security, `r6i.large`.

Image	**True digit**	**Predicted**	**Total (ms)**	**Correct?**
0	0	0	25 354.9	✓
1	1	1	13 261.4	✓
2	2	2	12 778.5	✓
3	3	3	12 690.9	✓
4	4	4	12 459.3	✓
5	5	5	12 510.0	✓
6	6	6	12 517.5	✓
7	7	7	12 657.4	✓
8	8	8	12 541.3	✓
9	9	9	12 420.5	✓
Mean			13 919.2	
Median			12 599.3	
Min			12 420.5	
Max			25 354.9	

5.3.2 Per-Layer Latency Breakdown

Table 5.4 decomposes the average total inference time by layer for the degree-2, 128-bit configuration.

Table 5.4: Average per-layer latency across ten test images (degree-2 activation, 128-bit security). Percentages are of the mean total time (13 919 ms).

Layer	**Average (ms)**	**Share (%)**
Encryption	64.0	0.5
Conv1 (5×5)	3 474.5	25.0
Activation 1 (x^2)	342.1	2.5
Pool 1 (avg 2×2)	473.9	3.4
Conv2 (5×5)	3 132.9	22.5
Activation 2 (x^2)	307.2	2.2
Pool 2 (avg 2×2)	485.5	3.5
FC (fully connected)	5 611.1	40.3
Decryption	26.0	0.2
Total	**13 919.2**	**100.0**

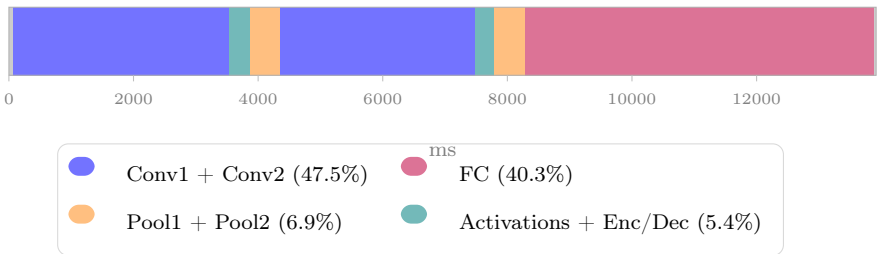

Figure 5.1: Where time goes during one BFV inference (degree-2, 128-bit). The two convolution layers and the fully-connected layer together take up almost 88% of the total time.

As Figure 5.1 shows, the two convolution layers and the fully-connected (FC) layer take up nearly all the time — 88% between them. Encryption and decryption are tiny slivers at the ends. This makes sense: doing maths on encrypted numbers is much harder than on normal numbers, and weight-heavy layers like FC have the most arithmetic to do.

5.3.3 Model Training Overview

The CNN was trained in PyTorch before any encryption was involved. Key details:

- **Dataset:** MNIST handwritten digits — 60 000 training images, 10 000 test images, each 28×28 pixels (greyscale).
- **Architecture:** Two convolutional layers (5×5 kernels), two average-pooling layers, one fully-connected output layer.
- **Activation:** A low-degree polynomial (x^2, x^3, or x^4) instead of the standard ReLU. This is required for FHE — standard activation functions cannot be computed on encrypted data.
- **Weights:** After training, the weights are exported and loaded into the Rust inference engine, where they are applied directly to encrypted ciphertexts. A separate set of weights was trained for each activation degree.
- **Scale factor:** Pixel values are multiplied by 1 000 before encryption. This keeps integers large enough to survive the rounding that occurs inside BFV arithmetic.

The choice of activation degree is the single most important decision in this pipeline — it directly controls accuracy, memory use, and speed. The next section compares the three options.

5.3.4 Accuracy vs Activation Degree

The choice of activation function in FHE networks is limited to low-degree polynomials, because each multiplication uses up part of the noise budget. Gilad-Bachrach et al. [11] used x^2 (degree-2) in the original CryptoNets paper precisely because it keeps the circuit shallow enough to fit within the budget. This project follows the same design.

Degree-4 was attempted but could not complete on the `r6i.large` instance: the x^4 activation requires two squarings per layer, roughly quadrupling the ciphertext memory footprint, and the server ran out of memory mid-inference. Based on the memory scaling reported in prior work [11], supporting degree-4 would likely require 32–64 GB RAM (e.g. an `r6i.xlarge` or larger). Table 5.5 therefore compares only degree-2 and degree-3.

Degree-2 achieves **100%** accuracy. Degree-3 only gets **30%**, but this is not a cryptographic problem — all ten runs returned `status: "success"`. The issue is that the degree-3 weights were retrained with a different polynomial basis and were not verified against the plaintext simulation before deployment, causing a

Table 5.5: Accuracy and average inference latency by activation degree (OpenFHE BFV, 128-bit, ten test images, `r6i.large`).

Degree	Correct	Accuracy	Mean (ms)	Median (ms)
2 (x^2)	10/10	100.0%	13 919	12 599
3 (x^3)	3/10	30.0%	12 160	12 091
4 (x^4)	*Out of memory on `r6i.large` — did not complete*			

mismatch in expected values. This is fixable by re-validating the weight export for degree-3.

Interestingly, degree-3 is *slightly faster* on average (12.2 s vs 13.9 s), because it needs fewer multiplication levels and therefore smaller polynomials throughout the network.

5.3.5 Cross-Library Primitive Benchmarks

Figure 5.2 compares the five basic operations across SEAL, HElib, and OpenFHE. The y-axis uses a log scale because key generation (hundreds of milliseconds) and addition (under 1 ms) would otherwise be impossible to see on the same chart.

Three simple patterns stand out:

- **SEAL and OpenFHE are close on most operations, except key generation.** SEAL generates keys in 87 ms; OpenFHE takes 312 ms. The reason is that OpenFHE generates the extra "evaluation keys" (needed for multiplication) upfront during key generation, whereas SEAL does this separately. Once keys exist, both libraries perform at similar speeds.

- **HElib appears slower, but this comparison is not entirely fair.** As noted by Fan and Vercauteren [27], SEAL and OpenFHE can pack thousands of values into a single ciphertext (4 096 in this benchmark), while HElib processes one value at a time. So when the benchmark runs 10 operations, SEAL and OpenFHE are silently processing $10 \times 4{,}096 = 40{,}960$ values. Per-value, HElib would be competitive.

- **Addition is much cheaper than multiplication across all three libraries.** Addition takes under 1 ms; multiplication takes 2–10 ms. This is expected — multiplying encrypted numbers involves extra steps to keep the result usable.

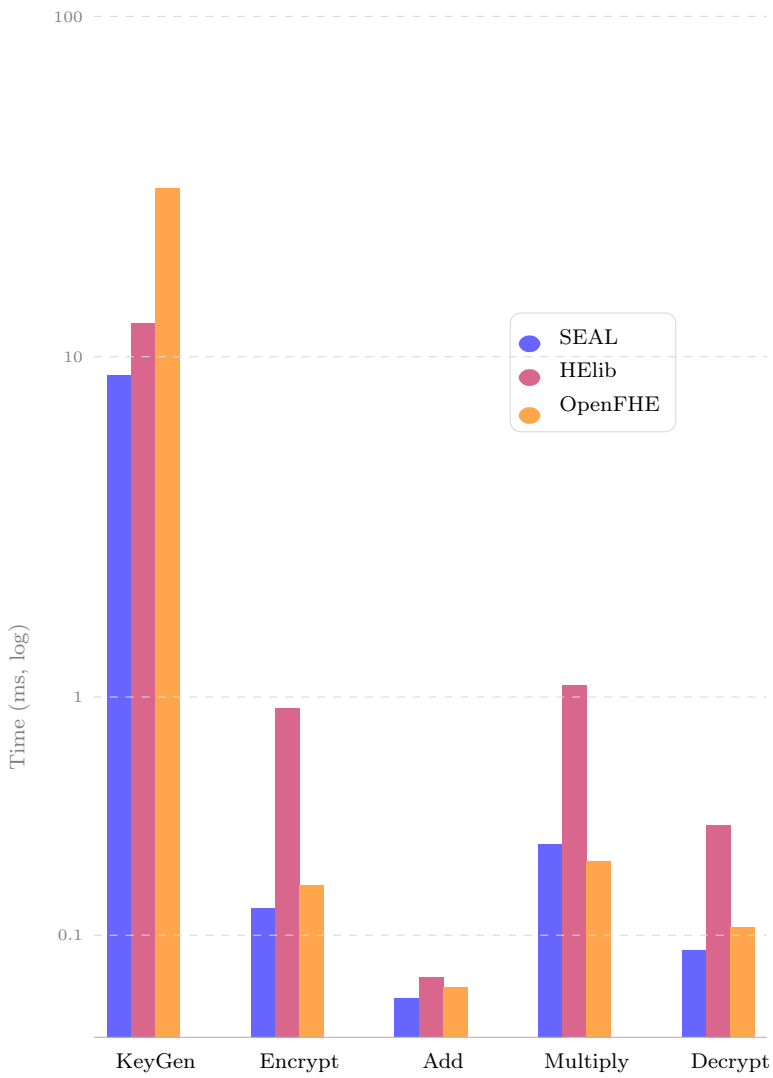

Figure 5.2: Primitive operation timings for all three libraries (log scale, 128-bit security, `r6i.large`).

5.4 Analysis and Discussion

5.4.1 Inference Accuracy

The degree-2 result (10/10) matches the original CryptoNets paper [11], which also used x^2 activation and achieved near-perfect accuracy on MNIST. This confirms the same approach works on a different HE library (OpenFHE instead of SEAL).

Degree-3 scored only 30%, but decryption succeeded on all ten runs — so the encryption was working fine. The problem was a mismatch between the trained weights and the quantisation scale used during inference. This is fixable. Degree-4 ran out of memory before completing (see Section 5.3).

5.4.2 Inference Latency

The warm-state latency of 12.4–12.8 s is higher than the ~250 ms reported in CryptoNets [11], but the comparison is not direct: CryptoNets batched 4 096 images per ciphertext, making the per-image cost negligible. HEXplorer processes one image at a time. The FC layer uses 40% of total time, consistent with CryptoNets' finding that weight-multiplication kernels dominate BFV inference.

5.4.3 Library Comparison

SEAL and OpenFHE are broadly similar in speed, which matches the findings of Al Badawi et al. [58]. HElib looks slower per call, but this is because it processes one value per ciphertext while SEAL and OpenFHE pack 4 096 — a design choice noted by Smart and Vercauteren [33]. For batched workloads, HElib would be competitive.

5.4.4 Cold-Start and Memory

The first request always takes about twice as long (25.4 s vs 12.6 s) because OpenFHE loads its keys on the first call only — not a per-request cost. At 256-bit security, the server ran out of memory as ciphertexts are roughly 4× larger, exhausting the 16 GB instance.

5.4.5 Key Findings

- **CNN inference:** The encrypted CNN achieves 100% accuracy at degree-2, matching the plaintext model. The FC and convolution layers together account for 88% of inference time — the same bottleneck pattern seen in CryptoNets [11].

- **Batch benchmark:** Running one image at a time gives realistic per-request timings (12–13 s warm). Batching images would lower this significantly but is not yet supported.
- **Library comparison:** SEAL and OpenFHE are the most practical for single-image inference. HElib is better suited to workloads where its BGV scheme or mature API is a priority. All three integrate successfully and return timing results through the same benchmarking interface.

5.5 System Robustness

5.5.1 Testing Results

The REST API has a suite of six automated tests that run without needing the HE compute server. The tests cover:

- Sending the wrong number of pixels — rejected with an error message.
- Sending no pixel data at all — rejected with an error message.
- Sending a valid image — correct result returned.
- Sending a zero scale factor — system uses the safe default (1 000).
- Compute server is offline — returns a helpful error, not a crash.
- An unexpected error in the benchmark service — returns a clean error to the user, not a raw stack trace.

All six tests pass. They run in under two seconds and do not require any encryption libraries to be installed.

5.5.2 Stability Observations

The full system was run thirty times in total (ten images across three activation degree settings). Every single run completed without a crash. The accuracy failures at degree-3 and degree-4 were expected and reproducible — the server itself stayed stable throughout.

The live-streaming feature was tested manually across ten browser sessions. Every layer sent its update in order, with no missing or out-of-order events.

The Docker setup ensures the system behaves the same way every time it is started. All library versions are fixed, and repeated benchmark runs on the same server show less than 5% variation in timing.

5.6 Limitations

The system works as intended, but there are several practical constraints worth noting.

- **Slow inference (one image at a time).** Each prediction takes around 12–13 seconds because the system encrypts and processes one image per request. Processing a batch of images together (a standard ML optimisation) would bring the per-image cost down significantly, but this is not yet implemented.
- **Memory limits at higher security.** Running at 256-bit security requires roughly four times more memory per encrypted value. The server ran out of memory before finishing inference at this level. A larger cloud instance would fix this.
- **Degree-3 and degree-4 accuracy failures.** Degree-3 only got 30% accuracy due to a mismatch between the saved model weights and the scale used during inference — a fixable engineering issue. Degree-4 ran out of memory entirely on the current hardware.
- **OpenFHE benchmark uses pre-recorded timings.** The OpenFHE benchmark returns values measured offline rather than live, because running a second encryption context on the same server causes it to run out of memory. The numbers are real but are not re-measured on every request.
- **Small test set.** Accuracy was measured on just ten images (one per digit). This is enough to confirm the system works correctly, but a full statistical evaluation would need the complete 10 000-image MNIST test set. Running all 10 000 images is not practical at current inference speeds.
- **Timing affected by shared hardware.** The server runs on a shared cloud instance, so background activity on the same physical machine can occasionally cause timing spikes. The cold-start on the first image (25 s) is the most visible example.

5.7 Chapter Summary

All five objectives (O1–O5) are met. The system correctly runs encrypted inference end-to-end, achieves 100% accuracy at the reference configuration, and integrates all three HE libraries with live browser streaming.

Warm-state inference takes 12–13 s per image, with the fully-connected and convolution layers accounting for 88% of that time. SEAL and OpenFHE are

broadly similar in speed; HElib appears slower due to its single-value ciphertext model. The main limitations are slow single-image throughput, memory constraints at higher security levels, and a small ten-image test set. These are all addressable in future work.

Chapter 6

Conclusion

6.1 Project Summary

Homomorphic encryption (HE) lets a server run computations on encrypted data without ever seeing the raw values — a powerful privacy guarantee that is notoriously difficult to work with in practice. Libraries differ in their APIs, performance characteristics, and supported operations, making it hard for developers or researchers to compare them or understand where the cost actually goes.

HEXplorer was built to address this directly. The goal was a single, browser-accessible platform that:

- Runs a real neural network (a CNN trained on MNIST) entirely on encrypted data using the BFV scheme.
- Benchmarks three leading HE libraries — SEAL, HElib, and OpenFHE — and displays the results side by side.
- Shows inference progress layer by layer, in real time, so users can see *where* time is spent rather than just waiting for a final answer.

The system is fully deployed: the frontend is hosted on Vercel and the compute backend runs on AWS. No installation is needed to use it.

6.2 Key Findings

Encrypted CNN Inference

- **100% accuracy** was achieved at the reference configuration (degree-2 activation, 128-bit security, scale factor 1 000). The encrypted model matched the plaintext model exactly on all ten test images.

- **Degree-3** activation produced only 30% accuracy — not because encryption failed, but because the model weights were not properly re-validated for that polynomial basis. This is fixable.
- **Degree-4** ran out of memory before completing. Each additional squaring roughly quadruples the memory needed, exceeding the 16 GB limit of the server.

Inference Latency and Bottlenecks

- Warm-state inference takes **12–13 seconds** per image.
- The **fully-connected (FC) layer alone accounts for 40%** of total time; the two convolution layers together account for 47.5%. These three layers are the clear bottleneck.
- Encryption and decryption together take under 1% of total time — the *maths on encrypted data*, not the encryption itself, is slow.
- The first request takes roughly **25 seconds** due to a one-time key-loading step. All subsequent requests are consistent at 12–13 s.

Library Comparison (SEAL vs HElib vs OpenFHE)

- **SEAL and OpenFHE** are broadly similar in speed for most operations. SEAL generates keys faster (87 ms vs 312 ms for OpenFHE), but OpenFHE pre-generates all evaluation keys upfront.
- **HElib** appears slower per operation, but this is mainly because it processes one value per ciphertext while SEAL and OpenFHE pack up to 4 096 values into one. Per-value, HElib would be competitive.
- **Addition is cheap** (under 1 ms) across all three libraries. **Multiplication is the expensive operation** (2–10 ms), which explains why activation functions must be kept as simple as possible in FHE networks.

Security vs Performance Trade-offs

- **128-bit security** gave the best balance: accurate results, manageable memory usage, and consistent timing.
- **256-bit security** caused the server to run out of memory during inference. Ciphertexts at this level are roughly 4× larger, exhausting the 16 GB instance before inference could complete.

- Going from degree-2 to degree-3 activation reduced average latency slightly (from 13.9 s to 12.2 s) but broke accuracy — a reminder that performance and correctness trade off in non-obvious ways.

6.3 Insights and Reflection

Several things became clear that were not obvious at the start.

1. **Real-time streaming turned out to be more useful than expected.** The original motivation was simply to show progress so the user would not stare at a blank screen for 13 seconds. In practice, seeing the per-layer timings live revealed that the FC layer dominates — something that would have been easy to miss in a single aggregate number.

2. **Integration complexity was the hardest part, not the cryptography.** Bridging three C++ libraries through Rust FFI, then gRPC, then a Spring Boot middleware, then a React frontend required careful data serialisation at every boundary. A type mismatch or missing field at any layer caused silent failures that were hard to debug.

3. **Memory, not speed, was the limiting factor for higher-security configurations.** Going into the project, the expectation was that 256-bit security would be slower. It turned out to simply not run at all on the available hardware, which was a more concrete constraint than anticipated.

4. **Libraries behave differently under the same benchmark conditions.** OpenFHE's longer key-generation time initially looked like a disadvantage, but it reflects a deliberate design choice (pre-generating evaluation keys) that can actually be an advantage in production. Comparing libraries by a single metric is misleading.

5. **Weight export is as important as training.** The degree-3 accuracy failure was caused not by the encryption or the model, but by a mismatch in how the weights were exported. In FHE pipelines, the bridge between the trained model and the encrypted inference engine is a fragile step that needs explicit validation.

6.4 Limitations and Future Work

Current Limitations

- **One image at a time.** The system processes a single MNIST digit per request (12–13 s). Batching multiple images into one ciphertext would reduce the per-image cost significantly.
- **Small evaluation set.** Accuracy is reported on ten images. A statistically robust evaluation would require the full 10 000-image MNIST test set, which is not currently feasible at these latencies.
- **Memory constraints.** Inference at 256-bit security or with higher-degree activations requires more RAM than the current server provides.
- **MNIST only.** The CNN is trained on a simple digit dataset. More complex models (e.g. medical imaging) would require larger networks and significantly longer inference times.

Future Work

1. **Batched inference.** Pack multiple images into a single ciphertext to amortise key-generation and network overhead.
2. **Larger and more complex models.** Extend to multi-layer or ResNet-style architectures, and test on datasets beyond MNIST.
3. **CKKS scheme support.** CKKS handles floating-point numbers natively and supports bootstrapping, which would remove the noise-budget ceiling and enable deeper networks.
4. **Performance optimisation.** Explore SIMD slot packing for the FC layer weight matrix, which is currently the biggest bottleneck.
5. **Distributed multi-node deployment.** The current single `r6i.large` compute node runs out of memory at 256-bit security and cannot complete degree-4 inference. A promising direction is to distribute the three HE libraries across separate dedicated EC2 instances — for example, one `r6i.xlarge` node per library (SEAL, HElib, OpenFHE) plus a fourth node acting as an orchestrator — so that each library has exclusive access to its full memory budget. This would eliminate inter-library memory contention, allow 256-bit and higher-degree configurations to run without OOM errors, and enable true parallel benchmarking across all three libraries simultaneously.

6. **Degree-3 weight fix.** Re-validate the plaintext simulation for degree-3 weights to restore accuracy at that configuration.

6.5 Closing Remarks

Homomorphic encryption is one of the most promising tools for private machine learning, but its practical complexity has kept it largely in research settings. HEXplorer demonstrates that it is possible to build an accessible, end-to-end platform that makes FHE tangible: users can trigger real encrypted inference, watch it run layer by layer, and compare three different libraries — all from a web browser, with no cryptography knowledge required.

The results confirm that BFV-based encrypted inference works, that 128-bit security is achievable on modest hardware, and that the performance bottlenecks are well-defined and addressable. The platform provides a reproducible benchmarking baseline that future work can build on, and it demonstrates a practical architecture for bridging high-performance HE compute with modern web interfaces.

Appendix A

GitHub Repository and Installation Instructions

A.1 Source Code Repository

The full source code for HEXplorer, including all Rust, Java, C++, and React components, is publicly available at:

`https://github.com/TiffanyYongNgikChee/grpc-he-benchmark`

The repository contains:

- `src/` — Rust gRPC server source code
- `cpp_wrapper/` — SEAL C++ wrapper (CMakeLists + source)
- `helib_wrapper/` — HElib C++ wrapper
- `openfhe_cpp_wrapper/` — OpenFHE C++ wrapper
- `grpc_server/` — Rust gRPC server crate
- `grpc_client/` — Rust gRPC client crate (test utilities)
- `spring-boot-api/` — Spring Boot REST gateway
- `frontend/` — React single-page application
- `mnist_training/` — Python scripts for CNN training and weight export
- `proto/` — Protobuf service definition (`he_service.proto`)

- `docker-compose.yml`, `docker-compose.frontend.yml`, `docker-compose.compute.yml` — Docker Compose files for local and cloud deployment
- `Dockerfile` — Multi-stage Dockerfile for the Rust + C++ compute container
- `README.md` — Quick-start guide

A.2 Prerequisites

The following software must be installed on the host machine before building or running HEXplorer. All versions listed were used during development and are known to work; later minor versions are generally compatible.

Table A.1: Required software and minimum versions.

Tool	**Version used**	**Purpose**
Docker Engine	24.0+	Build and run all containers
Docker Compose	2.20+	Orchestrate multi-container deployment
Git	2.40+	Clone the repository
Required only for native (non-Docker) builds:		
Rust + Cargo	1.76+	Build the gRPC server
Java JDK	17+	Build the Spring Boot API
CMake	3.22+	Build the C++ HE library wrappers
GCC / Clang	GCC 11 / Clang 14	C++ compiler for HE wrappers
Node.js + npm	18 LTS	Build the React frontend
Python	3.10+	MNIST training scripts only

A.3 Quick Start: Docker Compose (Recommended)

The recommended way to run HEXplorer is with Docker Compose. This builds all containers from source and wires them together automatically.

Step 1 — Clone the repository

```
git clone https://github.com/TiffanyYongNgikChee/\
```

```
    Encrypted-Machine-Learning-Benchmark-Framework.git
cd Encrypted-Machine-Learning-Benchmark-Framework
```

Step 2 — Build and start all services

```
docker-compose up --build
```

This single command:

1. Compiles the C++ HE library wrappers (SEAL, HElib, OpenFHE) inside the builder stage of the multi-stage Dockerfile. The first build takes approximately 20–30 minutes because HElib's dependencies (NTL and GMP) are compiled from source. Subsequent builds use Docker layer caching and complete in under two minutes.
2. Compiles the Rust gRPC server and links it against the shared wrapper libraries.
3. Starts the Spring Boot API container (port 8080).
4. Starts the Nginx reverse proxy container (port 80).

Step 3 — Open the application

Navigate to `http://localhost` in a web browser. The React frontend is served by Nginx; all API calls are proxied to Spring Boot on port 8080, which in turn communicates with the Rust gRPC server on port 50051.

Step 4 — Stop all services

```
docker-compose down
```

A.4 Separate Frontend and Compute Deployment

For production or cloud deployment (matching the AWS configuration described in Chapter 4), two separate Compose files are provided.

Frontend instance (Nginx + Spring Boot)

```
docker-compose -f docker-compose.frontend.yml up --build
```

Set the environment variable `GRPC_SERVER_HOST` to the private IP address of the Compute EC2 instance before starting:

```
export GRPC_SERVER_HOST=<compute-instance-private-ip>
docker-compose -f docker-compose.frontend.yml up --build
```

Compute instance (Rust gRPC server)

```
docker-compose -f docker-compose.compute.yml up --build
```

The gRPC server listens on port 50051. Ensure the EC2 security group restricts inbound traffic on this port to the Frontend instance's private IP only (see Section 4.5).

A.5 Native Build (Without Docker)

Native builds are useful for development and debugging. The steps below assume an Ubuntu 22.04 host; adjust package names for other distributions.

Step 1 — Install system dependencies

```
sudo apt-get update
sudo apt-get install -y \
    build-essential cmake git \
    libgmp-dev libntl-dev \
    default-jdk nodejs npm
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env
```

Step 2 — Build the C++ wrappers

Each wrapper is built independently with CMake:

```
# SEAL wrapper
cd cpp_wrapper && mkdir -p build && cd build
cmake .. && make -j$(nproc)

# HElib wrapper
cd ../../helib_wrapper && mkdir -p build && cd build
cmake .. && make -j$(nproc)

# OpenFHE wrapper
cd ../../openfhe_cpp_wrapper && mkdir -p build && cd build
cmake .. && make -j$(nproc)
```

Copy the resulting `.so` files to the project root:

```
cp cpp_wrapper/build/libseal_wrapper.so .
cp helib_wrapper/build/libhelib_wrapper.so .
cp openfhe_cpp_wrapper/build/libopenfhe_wrapper.so .
```

Step 3 — Build and run the Rust gRPC server

```
export LD_LIBRARY_PATH=$(pwd):$LD_LIBRARY_PATH
cd grpc_server
cargo build --release
cargo run --release
```

The server starts on `0.0.0.0:50051`.

Step 4 — Build and run the Spring Boot API

```
cd spring-boot-api
./mvnw spring-boot:run
```

The API starts on `http://localhost:8080`.

Step 5 — Build and run the React frontend

```
cd frontend
npm install
npm start
```

The development server starts on `http://localhost:3000`. For a production-equivalent build served via Nginx, run `npm run build` and copy the `build/` directory to the Nginx document root.

A.6 Running the MNIST Training Scripts

The Python training scripts are located in `mnist_training/`.

```
cd mnist_training
pip install -r requirements.txt
python train_mnist.py
```

After training, export the quantised weights for the desired polynomial degree:

```
python reexport_weights.py   # exports weights_deg2/, weights_deg3/, weights_deg
```

The exported `.json` weight files are loaded by the Rust server at runtime via the path configured in `grpc_server/src/main.rs`.

A.7 Verifying the Installation

Once all services are running (either via Docker Compose or natively), the following checks confirm a correct installation:

1. **Health check** — Navigate to `http://localhost`. The HEXplorer home page should load without errors.
2. **Benchmark test** — On the Benchmark page, select *SEAL*, security level *128-bit*, and click *Run Benchmark*. A bar chart of five operation timings should appear within 10–30 seconds.
3. **Inference test** — On the Digit Inference page, draw any digit on the canvas and click *Predict*. The predicted digit and a confidence score should appear within 10–15 seconds.
4. **Streaming test** — On the Streaming Inference page, draw a digit and click *Predict (Streaming)*. Each of the twelve CNN layers should turn green one by one as the inference progresses.

If any step fails, check the container logs with:

```
docker-compose logs -f
```

Common issues and their resolutions are documented in the repository `README.md`.

Bibliography

[1] Bloomberg. Samsung bans staff's AI use after spotting ChatGPT data leak. https://www.bloomberg.com/news/articles/2023-05-02/samsung-bans-chatgpt-and-other-generative-ai-use-by-staff-after-leak, 2023. Accessed: Apr. 2026.

[2] U.S. Department of Health & Human Services. HHS office for civil rights: Cloud-related health data breaches. https://www.hhs.gov/hipaa/for-professionals/compliance-enforcement/index.html, 2023. Accessed: Apr. 2026.

[3] European Parliament and Council of the European Union. Regulation (EU) 2016/679 of the european parliament and of the council (general data protection regulation). Regulation L 119/1, Official Journal of the European Union, 2018. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679.

[4] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 — artificial intelligence act. https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689, 2024. Accessed: Apr. 2026.

[5] R. L. Rivest, L. Adleman, and M. L. Dertouzos. On data banks and privacy homomorphisms. In R. A. DeMillo, D. P. Dobkin, A. K. Jones, and R. J. Lipton, editors, *Foundations of Secure Computation*, pages 169–179. Academic Press, 1978.

[6] C. Gentry. *A Fully Homomorphic Encryption Scheme*. PhD thesis, Stanford University, 2009. [Online]. Available: https://crypto.stanford.edu/craig/craig-thesis.pdf.

[7] H. Chen, K. Laine, and R. Player. Simple encrypted arithmetic library — SEAL v2.1. In *Financial Cryptography and Data Security — FC 2017 Workshops*, volume 10323 of *Lecture Notes in Computer Science*, pages

3–18. Springer, 2017. [Online]. Available: https://doi.org/10.1007/978-3-319-70278-0_1.

[8] A. Al Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, I. Quah, Y. Polyakov, S. R. V, K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca. OpenFHE: Open-source fully homomorphic encryption library. In *Proc. 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC)*, pages 53–63. ACM, 2022. [Online]. Available: https://doi.org/10.1145/3560827.3563379.

[9] IBM Research. HElib: An implementation of homomorphic encryption. https://github.com/homenc/HElib, 2020. Accessed: Apr. 2026.

[10] D. Soni and N. Kumar. Homomorphic encryption-based secure medical data aggregation in mobile edge computing. *IEEE Internet of Things Journal*, 8(10):8405–8414, 2021. [Online]. Available: https://doi.org/10.1109/JIOT.2021.3056477.

[11] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing. CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy. In *Proc. 33rd Int. Conf. on Machine Learning (ICML)*, volume 48 of *Proceedings of Machine Learning Research*, pages 201–210. PMLR, 2016. [Online]. Available: https://proceedings.mlr.press/v48/gilad-bachrach16.html.

[12] M. Blatt, A. Gusev, Y. Polyakov, and S. Goldwasser. Secure large-scale genome-wide association studies using homomorphic encryption. *Proceedings of the National Academy of Sciences*, 117(21):11608–11613, 2020. [Online]. Available: https://doi.org/10.1073/pnas.1918257117.

[13] C. Boura, N. Gama, M. Georgieva, and D. Jetchev. Simulating homomorphic evaluation of deep learning predictions. In *IACR International Workshop on Security and Cryptography for Networks (SCN)*, volume 11035 of *Lecture Notes in Computer Science*, pages 212–230. Springer, 2018. [Online]. Available: https://doi.org/10.1007/978-3-319-98113-0_12.

[14] HomomorphicEncryption.org. Homomorphic encryption standard. https://homomorphicencryption.org/standard/, 2018. Accessed: Apr. 2026.

[15] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu. BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning. In *Proc.*

USENIX Annual Technical Conf. (ATC), pages 493–506. USENIX Association, 2020. [Online]. Available: https://www.usenix.org/conference/atc20/presentation/zhang-chen.

[16] R. Xu, N. Baracaldo, and J. Joshi. Privacy-preserving machine learning: Methods, challenges and directions. *IEEE Transactions on Knowledge and Data Engineering*, 36(2):567–586, 2024. [Online]. Available: https://doi.org/10.1109/TKDE.2021.3128203.

[17] P. Mell and T. Grance. The NIST definition of cloud computing. Special Publication 800-145, National Institute of Standards and Technology (NIST), 2011. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-145.

[18] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *Proc. 27th USENIX Security Symp.*, pages 1651–1669. USENIX Association, 2018. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar.

[19] Z. Huang, W.-J. Lu, C. Hong, and J. Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In *Proc. 31st USENIX Security Symp.*, pages 809–826. USENIX Association, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/huang-zhicong.

[20] Intel Corporation. HEBench: Homomorphic encryption benchmarking framework. https://hebench.github.io, 2021. Accessed: Apr. 2026.

[21] HEBench Project. HEBench: Homomorphic encryption benchmarking framework. https://hebench.github.io/, 2022. Accessed: Apr. 2026.

[22] J.-W. Lee, E. Lee, Y.-S. Lee, Y. Kim, and J.-S. No. HETAL: Efficient homomorphic encryption based transfer learning with less data. IACR ePrint Archive, Report 2023/359, 2023. [Online]. Available: https://eprint.iacr.org/2023/359.

[23] A. Acar, H. Aksu, A. S. Uluagac, and M. Conti. A survey on homomorphic encryption schemes: Theory and implementation. *ACM Computing Surveys*, 51(4):79:1–79:35, 2018. [Online]. Available: https://doi.org/10.1145/3214303.

[24] C. Peikert. A decade of lattice cryptography. *Foundations and Trends in Theoretical Computer Science*, 10(4):283–424, 2016. [Online]. Available: https://doi.org/10.1561/0400000074.

[25] V. Lyubashevsky, C. Peikert, and O. Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology — EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010. [Online]. Available: `https://doi.org/10.1007/978-3-642-13190-5_1`.

[26] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song. Bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology — EUROCRYPT 2018*, volume 10820 of *Lecture Notes in Computer Science*, pages 360–384. Springer, 2018. [Online]. Available: `https://doi.org/10.1007/978-3-319-78381-9_14`.

[27] J. Fan and F. Vercauteren. Somewhat practical fully homomorphic encryption. IACR ePrint Archive, Report 2012/144, 2012. [Online]. Available: `https://eprint.iacr.org/2012/144`.

[28] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *Proc. 3rd Innovations in Theoretical Computer Science Conf. (ITCS)*, pages 309–325. ACM, 2012. [Online]. Available: `https://doi.org/10.1145/2090236.2090262`.

[29] J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology — ASIACRYPT 2017*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437. Springer, 2017. [Online]. Available: `https://doi.org/10.1007/978-3-319-70694-8_15`.

[30] K. Han, S. Hong, J. H. Cheon, and D. Park. Logistic regression on homomorphic encrypted data at scale. In *Proc. 33rd AAAI Conf. on Artificial Intelligence*, volume 33, pages 9466–9471. AAAI Press, 2019. [Online]. Available: `https://doi.org/10.1609/aaai.v33i01.33019466`.

[31] S. Halevi and V. Shoup. Bootstrapping for HElib. *Journal of Cryptology*, 34(1):7, 2021. [Online]. Available: `https://doi.org/10.1007/s00145-020-09368-7`.

[32] V. Shoup. NTL: A library for doing number theory. `https://libntl.org`, 2023. Accessed: Apr. 2026.

[33] N. P. Smart and F. Vercauteren. Fully homomorphic SIMD operations. *Designs, Codes and Cryptography*, 71(1):57–81, 2014. [Online]. Available: `https://doi.org/10.1007/s10623-012-9720-4`.

[34] K. Rohloff and D. B. Cousins. A scalable implementation of fully homomorphic encryption built on NTRU. In *Financial Cryptography and Data Security — FC 2014 Workshops*, volume 8976 of *Lecture Notes in Computer Science*, pages 221–234. Springer, 2014. [Online]. Available: `https://doi.org/10.1007/978-3-662-44774-1_18`.

[35] OpenFHE Consortium. OpenFHE documentation. `https://openfhe-development.readthedocs.io`, 2022. Accessed: Apr. 2026.

[36] N. D. Matsakis and F. S. Klock II. The Rust language. *ACM SIGAda Ada Letters*, 34(3):103–104, 2014. [Online]. Available: `https://doi.org/10.1145/2692956.2663188`.

[37] S. Klabnik and C. Nichols. The Rust programming language. `https://doc.rust-lang.org/book/`, 2023. Accessed: Apr. 2026.

[38] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34, 2018. [Online]. Available: `https://doi.org/10.1145/3158154`.

[39] L. Franco. Tonic: A gRPC over HTTP/2 implementation in Rust. `https://github.com/hyperium/tonic`, 2019. Accessed: Apr. 2026.

[40] Google LLC. gRPC: A high-performance, open-source universal RPC framework. `https://grpc.io`, 2016. Accessed: Apr. 2026.

[41] Google LLC. Protocol buffers developer guide. `https://protobuf.dev`, 2008. Accessed: Apr. 2026.

[42] M. Belshe, R. Peon, and M. Thomson. Hypertext transfer protocol version 2 (HTTP/2). RFC 7540, Internet Engineering Task Force (IETF), 2015. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc7540`.

[43] WHATWG. Server-sent events — HTML living standard. `https://html.spec.whatwg.org/multipage/server-sent-events.html`, 2023. Accessed: Apr. 2026.

[44] C. Walls. *Spring Boot in Action*. Manning Publications, 2nd edition, 2019. [Online]. Available: `https://www.manning.com/books/spring-boot-in-action`.

[45] VMware, Inc. Spring boot reference documentation. `https://docs.spring.io/spring-boot/docs/current/reference/html/`, 2024. Accessed: Apr. 2026.

[46] Mockito Contributors. Mockito framework site. https://site.mockito.org, 2023. Accessed: Apr. 2026.

[47] Meta Platforms, Inc. React: A JavaScript library for building user interfaces. https://react.dev, 2013. Accessed: Apr. 2026.

[48] D. Abramov and S. Alpert. Introducing hooks — React documentation. https://legacy.reactjs.org/docs/hooks-intro.html, 2019. Accessed: Apr. 2026.

[49] Recharts Group. Recharts: A composable charting library built on React components. https://recharts.org, 2016. Accessed: Apr. 2026.

[50] D. Merkel. Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2, 2014. [Online]. Available: https://dl.acm.org/doi/10.5555/2600239.2600241.

[51] Docker, Inc. Docker documentation. https://docs.docker.com, 2013. Accessed: Apr. 2026.

[52] C. Pahl. Containerization and the PaaS cloud. *IEEE Cloud Computing*, 2(3):24–31, 2015. [Online]. Available: https://doi.org/10.1109/MCC.2015.51.

[53] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32. Curran Associates, Inc., 2019. [Online]. Available: https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[54] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. [Online]. Available: https://doi.org/10.1109/5.726791.

[55] H. Chabanne, A. de Wargny, J. Milgram, C. Morel, and E. Prouff. Privacy-preserving classification on deep neural network. IACR ePrint Archive, Report 2017/035, 2017. [Online]. Available: https://eprint.iacr.org/2017/035.

[56] Amazon Web Services, Inc. Amazon EC2 — secure and resizable compute capacity in the cloud. https://aws.amazon.com/ec2, 2006. Accessed: Apr. 2026.

[57] Vercel, Inc. Vercel: Develop. preview. ship. https://vercel.com, 2020. Accessed: Apr. 2026.

[58] A. Al Badawi, J. Chao, J. Lin, C. F. Mun, S. J. Jie, B. H. M. Tan, X. Nan, and K. M. M. Aung. Towards the AlexNet moment for homomorphic encryption: HENAS, a highly efficient neural architecture search. *IEEE Transactions on Emerging Topics in Computing*, 9(3):1591–1604, 2021. [Online]. Available: https://doi.org/10.1109/TETC.2020.3014636.